



ΔΙΕΘΝΕΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΤΗΣ ΕΛΛΑΔΟΣ

ΔΙΕΘΝΕΣ ΠΑΝΕΠΙΣΤΗΜΙΟ ΤΗΣ ΕΛΛΑΔΟΣ

ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΑΡΑΓΩΓΗΣ ΚΑΙ ΔΙΟΙΚΗΣΗΣ

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

**{Ανάπτυξη εφαρμογής REACT για ροές δεδομένων στη
βιομηχανία}**

{ΜΠΟΥΡΜΠΟΥΛΑΣ ΠΑΝΑΓΙΩΤΗΣ}

ΑΜ: 2020/134

ΕΠΙΒΛΕΠΩΝ

ΕΠΙΚΟΥΡΟΣ ΚΑΘΗΓΗΤΗΣ

ΔΗΜΗΤΡΙΟΣ ΜΠΕΧΤΣΗΣ

ΘΕΣΣΑΛΟΝΙΚΗ {2025}

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Διεθνούς Πανεπιστημίου Ελλάδος.

Υπεύθυνη Δήλωση Συγγραφέα:

Δηλώνω ρητά ότι, σύμφωνα με το άρθρο 8 του Ν. 1599/1986 και τα άρθρα 2,4,6 παρ. 3 του Ν.1256/1982, η παρούσα εργασία αποτελεί αποκλειστικά προϊόν προσωπικής εργασίας και δεν προσβάλλει κάθε μορφής πνευματικά δικαιώματα τρίτων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον.

Μπούρμπουλας Παναγιώτης

(Υπογραφή)

Ευχαριστίες

Πρωτίστως θα ήθελα να ευχαριστήσω θερμά τον υπεύθυνο καθηγητή Δημήτριο Μπεχτσή για την απέραντη βοήθεια και καθοδήγησή του στην εκπόνηση της διπλωματικής μου εργασίας, και για τη μεγάλη στήριξη και εμπιστοσύνη που μου έδειξε κατά τη διάρκεια αυτής. Επίσης, στα υπόλοιπα μέλη της εξεταστικής επιτροπής για την προσεκτική ανάγνωση της εργασίας.

Θα ήθελα να εκφράσω ιδιαίτερες ευχαριστίες από καρδιάς τους γονείς μου για τη στήριξη και την εμπιστοσύνη που μου έδειξαν κατά τη διάρκεια εκπόνησης της παρούσας διπλωματικής εργασίας, όπως και για την αγάπη τους όλα αυτά τα χρόνια.

Περίληψη

Η παρούσα διπλωματική εργασία πραγματεύεται την ανάπτυξη μίας web εφαρμογής βασισμένης σε React για την παρακολούθηση και επεξεργασία βιομηχανικών ροών δεδομένων σε πραγματικό χρόνο. Στόχος είναι η δημιουργία ενός λειτουργικού εργαλείου που να επιτρέπει στους χρήστες να συλλέγουν, να αποθηκεύουν και να οπτικοποιούν δεδομένα από ποικίλες βιομηχανικές πηγές, αξιοποιώντας σύγχρονες τεχνολογίες διαδικτύου των πραγμάτων (IoT) και πρότυπα επικοινωνίας όπως MQTT, OPC UA και Modbus.

Αρχικά, παρουσιάζεται το θεωρητικό υπόβαθρο, με ανάλυση των τεχνολογιών ανάπτυξης λογισμικού, των αισθητήρων και των APIs που υποστηρίζουν την καταγραφή και διανομή δεδομένων. Έμφαση δίνεται στην αρχιτεκτονική της εφαρμογής, η οποία διαχωρίζεται σε frontend (React) και backend (Node.js/Express), ενώ ενσωματώνονται επιπλέον τεχνολογίες όπως Firebase για αποθήκευση cloud και RESTful APIs για τη διασύνδεση υποσυστημάτων.

Η υλοποίηση περιλαμβάνει μηχανισμούς επιλογής αισθητήρων από διάφορες πηγές (Arduino, TTN, Weather API), δυναμική εμφάνιση των δεδομένων μέσω γραφημάτων, και διαχείριση χρηστών με ασφαλή πρόσβαση. Το σύστημα υποστηρίζει τόσο την αποστολή δεδομένων όσο και την εξαγωγή τους σε μορφή CSV, καθιστώντας την εφαρμογή χρήσιμη για ανάλυση και αξιολόγηση σε βιομηχανικό περιβάλλον.

Τέλος, μέσα από πειραματικές δοκιμές και σενάρια χρήσης, αναλύονται τα αποτελέσματα λειτουργίας του συστήματος, εντοπίζονται περιορισμοί και προτείνονται βελτιώσεις. Η εργασία καταλήγει με συμπεράσματα σχετικά με την πρακτικότητα της εφαρμογής και τη δυνατότητα μελλοντικής επέκτασής της με λειτουργίες τεχνητής νοημοσύνης, μοντέλα πρόβλεψης, καθώς και ενίσχυση της διαλειτουργικότητας με επιπλέον πρωτόκολλα και βάσεις δεδομένων.

Abstract

This thesis focuses on the development of a web application using React for real-time monitoring and processing of industrial data streams. The goal is to create a functional tool that enables users to collect, store, and visualize data from various industrial sources, leveraging modern Internet of Things (IoT) technologies and communication protocols such as MQTT, OPC UA, and Modbus.

The work begins with an overview of the theoretical background, covering software development technologies, sensors, and data provision services (APIs) that support data acquisition and distribution. Emphasis is placed on the system architecture, which is divided into a frontend (React) and a backend (Node.js/Express), along with the integration of additional technologies like Firebase for cloud storage and RESTful APIs for subsystem interconnection.

The implementation includes mechanisms for selecting sensors from different sources (Arduino, TTN, Weather API), dynamic data visualization through graphs, and secure user access management. The system supports both real-time data streaming and CSV export, making it a useful tool for industrial analysis and evaluation.

Finally, through experimental testing and usage scenarios, the system's performance is evaluated, limitations are identified, and improvements are suggested. The thesis concludes with insights into the practicality of the application and its potential for future expansion with artificial intelligence features, predictive models, and enhanced interoperability with additional protocols and databases.

Contents

1	Εισαγωγή.....	8
1.1	Τεχνολογίες Ανάπτυξης Λογισμικού: Επιλογές Σήμερα	8
1.2	Αισθητήρια και Τεχνολογίες Διασύνδεσης.....	9
1.3	Υπηρεσίες Παροχής Δεδομένων (APIs)	11
2	Θεωρητικό Υπόβαθρο	13
2.1	Βάσεις Δεδομένων	13
2.2	Τεχνολογίες IoT.....	15
2.3	Πρωτόκολλα Επικοινωνίας	16
2.4	Εφαρμογές Παρακολούθησης στη Βιομηχανία.....	18
3	Αρχιτεκτονική Συστήματος	19
3.1	Ανάλυση Απαιτήσεων και Τεχνικών Προδιαγραφών.....	19
3.2	Συνολική Αρχιτεκτονική Συστήματος	20
3.3	Διάγραμμα Ροής και Αλληλεπίδρασης	23
3.4	Επιλογή Τεχνολογιών και Βιβλιοθηκών	26
3.5	Ασφάλεια και Επεκτασιμότητα της Υλοποίησης	28
4	Υλοποίηση Εφαρμογής.....	29
4.1	Δημιουργία και Αρχικοποίηση Έργου React	29
4.2	Ανάλυση και Χρήση Frontend Frameworks	30
4.3	Δομή και Οργάνωση Κώδικα στο Frontend.....	31
4.4	Ενσωμάτωση Backend – Node.js, Express και API	32
4.5	Βάσεις Δεδομένων και Αποθήκευση Δεδομένων.....	33
4.6	Διασύνδεση με IoT Πρωτόκολλα και Sensor Handlers	34
4.7	Χειρισμός Χαρτών και Γεωχωρικών Πληροφοριών	36
4.8	Οπτικοποίηση Δεδομένων – Γραφήματα και Μετρητές.....	37
4.9	Διαχείριση Χρηστών και Ασφάλεια	38
4.10	Τεχνητή Νοημοσύνη ως Εργαλείο Υποστήριξης στην Ανάπτυξη της Εφαρμογής	39
4.11	Συνολική Ενσωμάτωση Frameworks – Βιβλιοθήκες και Ρόλοι.....	39
4.12	Ενσωμάτωση με Docker	41
5	Πειραματική Δοκιμή του Συστήματος	42
5.1	Συμπεράσματα από τη Λειτουργία του Συστήματος.....	42
5.2	Προτάσεις για Μελλοντική Εργασία	42
5.3	Προοπτικές Εξέλιξης της Εφαρμογής.....	42
6	Βιβλιογραφία.....	46

Πίνακας εικόνων

Εικόνα 1 Διάγραμμα ροής λειτουργίας και αλληλεπίδρασης του συστήματος...	23
Εικόνα 2 Διεπαφή χρήστη με βασικές επιλογές στο αρχικό μενού.....	24
Εικόνα 3 Διεπαφή χρήστη με Register sensor.....	25
Εικόνα 4 Επιλογές Sensor.....	25
Εικόνα 5 Data στο Firebase.....	26
Εικόνα 6 Dashboard View Sensor.....	26

1 Εισαγωγή

1.1 Τεχνολογίες Ανάπτυξης Λογισμικού: Επιλογές Σήμερα

Η ανάπτυξη λογισμικού αποτελεί θεμέλιο λίθο για κάθε σύγχρονο ψηφιακό σύστημα, με τις διαθέσιμες τεχνολογίες να έχουν εξελιχθεί ραγδαία τις τελευταίες δεκαετίες. Σήμερα, οι προγραμματιστές διαθέτουν μια ευρεία γκάμα γλωσσών προγραμματισμού, frameworks, πλατφορμών και εργαλείων συνεχούς ενσωμάτωσης (CI/CD), που επιτρέπουν τη δημιουργία ευέλικτων, επεκτάσιμων και ασφαλών εφαρμογών. Οι τεχνολογίες αυτές διαχωρίζονται σε frontend, backend, βάσεις δεδομένων, APIs και cloud υποδομές, με κάθε κατηγορία να διαδραματίζει καθοριστικό ρόλο στη σύγχρονη ανάπτυξη.

Frontend Τεχνολογίες

Το frontend, υπεύθυνο για την απευθείας αλληλεπίδραση με τον χρήστη, βασίζεται σε frameworks και βιβλιοθήκες που εξασφαλίζουν δυναμικές και αποδοτικές διεπαφές. Η **React**, μια βιβλιοθήκη JavaScript που αναπτύχθηκε από το Facebook, κυριαρχεί λόγω της φιλοσοφίας των επαναχρησιμοποιήσιμων components και της αποτελεσματικής διαχείρισης κατάστασης (state management) μέσω εργαλείων όπως το Redux ή το Context API [1]. Παράλληλα, το **Vue.js** προβάλλεται για την ευκολία χρήσης και τη χαμηλή καμπύλη εκμάθησής του, ενώ το **Angular** (Google) προσφέρει ένα πλήρες οικοσύστημα για την ανάπτυξη μεγάλων εφαρμογών επιχειρησιακού επιπέδου, με ενσωματωμένα εργαλεία για testing και διαχείριση dependencies [2].

Backend Τεχνολογίες

Στο backend, όπου επικεντρώνεται η επιχειρησιακή λογική και η διαχείριση δεδομένων, οι επιλογές ποικίλλουν ανάλογα με τις απαιτήσεις κλιμακωσιμότητας και ασφάλειας. Το **Node.js**, με βάση το JavaScript, επιτρέπει ασύγχρονη, event-driven ανάπτυξη server-side εφαρμογών, συχνά σε συνδυασμό με το ελαφρύ framework **Express.js** για τη δημιουργία RESTful APIs [3]. Για εφαρμογές που απαιτούν ταχύτητα ανάπτυξης και απλότητα, τα frameworks **Django** και **Flask** (Python) ή το **Ruby on Rails** αποτελούν ιδανικές επιλογές, ιδίως σε περιβάλλοντα startups. Αντίθετα, για εταιρικές λύσεις με έμφαση στη διαλειτουργικότητα και την ασφάλεια, το **.NET Core** (Microsoft) και το **Java Spring Boot** προσφέρουν ευρεία υποστήριξη για υπηρεσίες middleware και ενσωματωμένες λύσεις διαχείρισης ταυτοτήτων [4].

Βάσεις Δεδομένων και Cloud Υποδομές

Η επιλογή βάσης δεδομένων καθορίζει την κλιμακωσιμότητα και την απόδοση μιας εφαρμογής. Οι σχεσιακές βάσεις (SQL) όπως η **MySQL** και η **PostgreSQL** είναι κατάλληλες για δομημένα δεδομένα με σταθερό σχήμα, ενώ οι βάσεις **NoSQL** (π.χ. **MongoDB**, **Firebase Realtime DB**) χρησιμοποιούνται σε εφαρμογές με real-time απαιτήσεις ή semi-structured δεδομένα, όπως σε συστήματα IoT. Παράλληλα, οι πλατφόρμες cloud (**AWS**, **Azure**, **Google Cloud**) έχουν γίνει

απαραίτητες για τη διαχείριση υποδομών, προσφέροντας αυτοματοποιημένη κλιμάκωση, παρακολούθηση (monitoring) και ενσωμάτωση με υπηρεσίες machine learning ή analytics [5].

DevOps, APIs και CI/CD

Η εφαρμογές πλέον απαιτούν συνεχή ενημέρωση και αξιοπιστία, γεγονός που έχει μετατρέψει τα εργαλεία **CI/CD** (GitHub Actions, GitLab CI, Jenkins) σε βασικά συστατικά της ανάπτυξης. Αυτά εξασφαλίζουν αυτόματη δοκιμή και ανάπτυξη νέων features, μειώνοντας τα errors και το χρόνο κυκλοφορίας. Επιπλέον, η χρήση **RESTful APIs** ή **GraphQL** διευκολύνει την επικοινωνία μεταξύ διαφορετικών συστημάτων, ιδίως σε βιομηχανικά περιβάλλοντα όπου απαιτείται η σύνδεση με αισθητήρες, μηχανήματα ή εξωτερικές υπηρεσίες. Στο πλαίσιο του IoT, αυτή η διασύνδεση γίνεται κρίσιμη για τη συλλογή και επεξεργασία δεδομένων σε πραγματικό χρόνο.

Με αυτές τις τεχνολογίες, οι developers μπορούν να αναπτύξουν λύσεις που ανταποκρίνονται στις συνεχώς εξελισσόμενες απαιτήσεις της ψηφιακής εποχής, διασφαλίζοντας ταυτόχρονα απόδοση, ασφάλεια και ευκολία συντήρησης.

1.2 Αισθητήρια και Τεχνολογίες Διασύνδεσης

Η χρήση αισθητήρων (sensors) αποτελεί θεμελιώδες στοιχείο της σύγχρονης τεχνολογίας, ιδιαίτερα σε τομείς όπως το Διαδίκτυο των Πραγμάτων (IoT), η βιομηχανική αυτοματοποίηση, η περιβαλλοντική παρακολούθηση και η έξυπνη γεωργία. Οι αισθητήρες λειτουργούν ως τα «αισθητήρια όργανα» των συστημάτων, συλλέγοντας δεδομένα από το φυσικό περιβάλλον—όπως θερμοκρασία, υγρασία, πίεση, φωτεινότητα, δόνηση ή ακόμα και χημικές ουσίες. Αυτά τα δεδομένα μετατρέπονται σε σήματα που μπορούν να επεξεργαστούν από μικροελεγκτές ή να αποσταλούν σε cloud πλατφόρμες για ανάλυση, επιτρέποντας τη λήψη αποφάσεων σε πραγματικό χρόνο. Η επιλογή του κατάλληλου αισθητήρα και της τεχνολογίας διασύνδεσής του είναι κρίσιμη για την αξιοπιστία και την αποτελεσματικότητα κάθε εφαρμογής.

Κατηγορίες Αισθητήρων

Οι αισθητήρες διακρίνονται σε τρεις βασικές κατηγορίες ανάλογα με τον τρόπο εξόδου και τις δυνατότητές τους. **Αναλογικοί αισθητήρες** (π.χ., θερμίστορ, LM35) παράγουν συνεχή σήματα, όπως μεταβαλλόμενη τάση, που αντιστοιχεί σε φυσικές μεταβολές (π.χ., αύξηση θερμοκρασίας). **Ψηφιακοί αισθητήρες**, όπως ο DHT11 (θερμοκρασία-υγρασία) ή ο MPU6050 (επιταχυνσιόμετρο), εξάγουν διακριτά δεδομένα (0/1) ή επικοινωνούν μέσω ψηφιακών διασυνδέσεων (I2C, SPI), προσφέροντας ακρίβεια και

ευκολία στην ενσωμάτωση με μικροελεγκτές. Σε αντίθεση, **έξυπνοι αισθητήρες** (smart sensors) συνδυάζουν λειτουργίες μέτρησης, επεξεργασίας και επικοινωνίας, συχνά με ενσωματωμένο μικροελεγκτή και υποστήριξη για πρωτόκολλα όπως το Modbus RTU, LoRa ή Zigbee. Αυτοί είναι ιδανικοί για απομακρυσμένες ή βιομηχανικές εφαρμογές, όπου απαιτείται αυτονομία και αξιοπιστία [6].

Μέθοδοι Διασύνδεσης Αισθητήρων

Η διασύνδεση αισθητήρων εξαρτάται από παράγοντες όπως η απόσταση, το εύρος ζώνης και οι περιβαλλοντικές συνθήκες. Για **ενσύρματα συστήματα**, τα πιο διαδεδομένα πρωτόκολλα περιλαμβάνουν:

- **UART (Serial):** Χαμηλού κόστους και απλό στη χρήση, χρησιμοποιείται σε πλατφόρμες όπως Arduino.
- **I2C και SPI:** Επιτρέπουν τη σύνδεση πολλών συσκευών σε έναν δίαυλο, με το SPI να προσφέρει υψηλότερη ταχύτητα.
- **RS-485/Modbus:** Βιομηχανικό πρότυπο για μεγάλες αποστάσεις (έως 1.2 km) και ανθεκτικότητα σε θόρυβο, ιδανικό για εργοστάσια ή γεωργικές εφαρμογές [6].

Στις **ασύρματες τεχνολογίες**, οι επιλογές καλύπτουν ένα ευρύ φάσμα αναγκών:

- **Wi-Fi (ESP8266/ESP32):** Ιδανικό για εφαρμογές εντός κτηρίων με απαιτήσεις υψηλής ταχύτητας.
- **Bluetooth/BLE:** Βελτιστοποιημένο για συσκευές χαμηλής κατανάλωσης (π.χ., wearable αισθητήρες).
- **LoRaWAN:** Καλύπτει εμβέλεια έως 10 km με ελάχιστη ενέργεια, κατάλληλο για αγροτικές ή περιβαλλοντικές εφαρμογές [7].
- **NB-IoT & LTE-M:** Βασίζονται σε δίκτυα κινητής τηλεφωνίας, προσφέροντας παγκόσμια κάλυψη και αξιοπιστία για εφαρμογές όπως έξυπνα μέτρα [8].
- **Ενσωμάτωση με Πλατφόρμες και Μικροελεγκτές**

Η ενσωμάτωση αισθητήρων απαιτεί τη χρήση πλατφορμών ανάπτυξης που εξασφαλίζουν ευελιξία και σταθερότητα. Για πειραματικές ή εκπαιδευτικές εφαρμογές, το **Arduino UNO/Mega** αποτελεί την καλύτερη επιλογή λόγω απλότητας και εκτενών βιβλιοθηκών. Για έξυπνες συσκευές με ασύρματη σύνδεση, τα **ESP32/ESP8266** προσφέρουν ενσωματωμένο Wi-Fi/Bluetooth και επεξεργαστική ισχύ για πρωτόκολλα όπως το MQTT. Σε πιο πολύπλοκες εφαρμογές, όπως η βιομηχανική αυτοματοποίηση, οι πλακέτες **STM32** ή **PLCs** (Programmable Logic Controllers) διασφαλίζουν αξιοπιστία υπό ακραίες συνθήκες. Επιπλέον, ο **Raspberry Pi** επιτρέπει την εκτέλεση πλήρους λειτουργικού συστήματος (Linux) για εφαρμογές που απαιτούν machine learning ή οπτικοποίηση δεδομένων σε πραγματικό χρόνο. Τα δεδομένα από τους αισθητήρες μπορούν να μεταδίδονται σε servers ή cloud πλατφόρμες (π.χ., AWS IoT, Azure IoT) μέσω πρωτοκόλλων όπως HTTP, MQTT, ή βιομηχανικών προτύπων όπως το OPC-UA [9].

Εφαρμογές στην Πράξη

Οι δυνατότητες των αισθητήρων εκδηλώνονται σε ποικίλους τομείς. Στην **έξυπνη γεωργία**, αισθητήρες υγρασίας και φωτεινότητας συνδέονται με αυτοματοποιημένα συστήματα άρδευσης για βελτιστοποίηση της χρήσης νερού. Σε **βιομηχανικά περιβάλλοντα**, αισθητήρες δόνησης ή θερμοκρασίας ενσωματώνονται σε μηχανήματα για predictive maintenance, μειώνοντας το κόστος διακοπής λειτουργίας. Επίσης, σε **περιβαλλοντικές εφαρμογές**, αισθητήρες ποιότητας αέρα ή νερού (π.χ., TDS για βαρέα μέταλλα) χρησιμοποιούνται για την ανίχνευση ρύπανσης σε πραγματικό χρόνο. Ο συνδυασμός τους με τεχνολογίες cloud και μηχανικής μάθησης επιτρέπει τη δημιουργία συστημάτων πρόβλεψης—για παράδειγμα, πρόγνωσης πλημμύρας ή βελτιστοποίησης ενεργειακής κατανάλωσης σε έξυπνα κτήρια.

Με την ταχεία ανάπτυξη του IoT και της βιομηχανίας 4.0, οι αισθητήρες και οι τεχνολογίες διασύνδεσής τους αποτελούν κρίσιμο εργαλείο για τη μετάβαση σε έξυπνα, δεδομενοκεντρικά συστήματα που βελτιώνουν την ποιότητα ζωής και την αποδοτικότητα.

1.3 Υπηρεσίες Παροχής Δεδομένων (APIs)

Οι Υπηρεσίες Παροχής Δεδομένων μέσω APIs αποτελούν κρίσιμο συστατικό των σύγχρονων εφαρμογών, ιδιαίτερα σε τομείς όπως το IoT. Τα APIs λειτουργούν ως διαμεσολαβητές μεταξύ διαφορετικών συστημάτων, επιτρέποντας την ανταλλαγή δεδομένων μεταξύ αισθητήρων, cloud πλατφορμών και εφαρμογών. Για παράδειγμα, ένας αισθητήρας θερμοκρασίας μπορεί να στέλνει δεδομένα σε ένα cloud σύστημα μέσω REST API, ενώ μια εφαρμογή μπορεί να τα ανακτά για οπτικοποίηση ή ανάλυση [10].

Ένα **API (Application Programming Interface)** είναι ένα σύνολο κανόνων που καθορίζουν πώς δύο συστήματα επικοινωνούν. Βασίζεται σε πρωτόκολλα όπως το HTTP για αιτήματα web ή το MQTT για συσκευές IoT χαμηλής ισχύος, με δεδομένα να αναπαρίστανται σε μορφές όπως JSON ή XML. Για παράδειγμα, ένα αίτημα GET σε ένα REST API μπορεί να επιστρέφει δεδομένα καιρού σε JSON μορφή, τα οποία στη συνέχεια χρησιμοποιούνται από μια εφαρμογή [10].

Οι κύριοι τύποι APIs περιλαμβάνουν τα **REST APIs**, που βασίζονται σε απλές HTTP μεθόδους (GET, POST, PUT, DELETE) και είναι ιδανικά για εφαρμογές IoT λόγω ευελιξίας [10], τα **SOAP APIs**, που χρησιμοποιούν XML για αυστηρά δομημένα μηνύματα και προτιμώνται σε εταιρικά συστήματα με αυξημένες απαιτήσεις ασφαλείας [11], τα **GraphQL APIs**, που επιτρέπουν στους χρήστες να καθορίζουν ακριβώς ποια δεδομένα θέλουν να λάβουν, μειώνοντας την υπερφόρτωση δικτύου [13], και τα **WebSockets**, που υποστηρίζουν real-time επικοινωνία, όπως live ροή δεδομένων από αισθητήρες [12].

Σε IoT εφαρμογές, τα APIs χρησιμοποιούνται τόσο για **λήψη** όσο και για **αποστολή** δεδομένων. Για παράδειγμα, το **OpenWeatherMap API** [14] παρέχει δεδομένα καιρού για έξυπνες γεωργικές εφαρμογές, ενώ αισθητήρες στέλνουν μετρήσεις σε πλατφόρμες όπως το **AWS IoT** ή το **Firebase** μέσω MQTT [10]. Πλατφόρμες όπως το **The Things Network (TTN)** προσφέρουν APIs για διαχείριση συσκευών LoRaWAN, επιτρέποντας την ομαλή ενσωμάτωση δεδομένων σε cloud συστήματα [10].

Η προστασία των APIs είναι κρίσιμη και απαιτεί τεχνικές όπως η χρήση **API Keys** ή **OAuth 2.0** για πιστοποίηση ταυτότητας, το **rate limiting** για αποφυγή κατάχρησης και η **κρυπτογράφηση TLS/SSL** για ασφαλή μεταφορά δεδομένων [10].

Ένα κλασικό παράδειγμα χρήσης API είναι το **OpenWeatherMap API**, όπου ένα αίτημα GET στην διεύθυνση:

GET

https://api.openweathermap.org/data/2.5/weather?q=Thessaloniki&appid=your_api_key

επιστρέφει δεδομένα σε μορφή JSON με πληροφορίες όπως θερμοκρασία και υγρασία [14]. Επίσης, το **Firebase Realtime Database** επιτρέπει αποθήκευση και συγχρονισμό δεδομένων πραγματικού χρόνου μέσω REST endpoints [14].

Τα οφέλη της χρήσης APIs είναι πολλαπλά. Παρέχουν **ευελιξία** μέσω της επαναχρησιμοποίησης υπηρεσιών (π.χ., χάρτες Google) χωρίς ανάπτυξη από το μηδέν, **κλιμάκωση** με εύκολη προσθήκη νέων λειτουργιών και **απόδοση** μέσω αποκεντρωμένης επεξεργασίας δεδομένων που μειώνει τον φόρτο εφαρμογών [10].

Μέσω των APIs, οι εφαρμογές IoT μετατρέπονται σε δυναμικά οικοσυστήματα που συνδέουν τον φυσικό με τον ψηφιακό κόσμο, ενισχύοντας την καινοτομία και τη βιωσιμότητα.

2 Θεωρητικό Υπόβαθρο

2.1 Βάσεις Δεδομένων

Οι βάσεις δεδομένων αποτελούν τον θεμέλιο λίθο των σύγχρονων συστημάτων πληροφορικής, καθώς επιτρέπουν την αποθήκευση, οργάνωση, ανάκτηση και διαχείριση μεγάλων όγκων δεδομένων με συστηματικό τρόπο. Η σημασία τους εκτείνεται σε πολυάριθμους τομείς, όπως τραπεζικά συστήματα, βιομηχανικοί αυτοματισμοί, ηλεκτρονικό εμπόριο, τηλεπικοινωνιακά δίκτυα, εφαρμογές υγείας και συστήματα IoT [15]. Μέσω αυτών, οι εφαρμογές μπορούν να διαχειρίζονται δεδομένα με τρόπο που διασφαλίζει ακρίβεια, ασφάλεια και ταχύτητα πρόσβασης, καθιστώντας τις απαραίτητες για τη λειτουργία σχεδόν κάθε ψηφιακής υπηρεσίας.

Ορισμός και Σκοπός

Μια βάση δεδομένων ορίζεται ως μια **οργανωμένη συλλογή δεδομένων** που σχετίζονται μεταξύ τους και αντικατοπτρίζουν μια πτυχή του πραγματικού κόσμου. Ο κύριος σκοπός της είναι να παρέχει έναν **αποδοτικό, ασφαλή και αξιόπιστο** τρόπο αποθήκευσης και πρόσβασης σε πληροφορίες. Για παράδειγμα, σε ένα νοσοκομείο, μια βάση δεδομένων μπορεί να αποθηκεύει ιατρικά ιστορικά ασθενών, ενώ σε ένα εμπορικό site, να διαχειρίζεται παραγγελίες και αποθέματα [16]. Η δομή της εξασφαλίζει ότι τα δεδομένα παραμένουν συνεπή, εύκολα αναζητήσιμα και προστατευμένα από απώλειες ή μη εξουσιοδοτημένη πρόσβαση.

Συστήματα Διαχείρισης Βάσεων Δεδομένων (DBMS)

Τα **Συστήματα Διαχείρισης Βάσεων Δεδομένων (DBMS)** είναι ειδικά λογισμικά που διευκολύνουν τη δημιουργία, ενημέρωση και διαχείριση βάσεων δεδομένων. Λειτουργούν ως διεπαφή μεταξύ των χρηστών/εφαρμογών και των δεδομένων, προσφέροντας λειτουργίες όπως εισαγωγή, τροποποίηση, αναζήτηση και διαγραφή πληροφοριών. Βασικά χαρακτηριστικά ενός DBMS περιλαμβάνουν την **ανεξαρτησία δεδομένων**, η οποία διαχωρίζει τη λογική της εφαρμογής από τη φυσική αποθήκευση, την **ακεραιότητα δεδομένων** μέσω περιορισμών και κανόνων, την **ασφάλεια** με έλεγχο πρόσβασης και κρυπτογράφηση, την **ανάκτηση από σφάλματα** με δημιουργία backups και την **διαχείριση ταυτόχρονης πρόσβασης** πολλών χρηστών χωρίς απώλεια συνοχής [17].

Τύποι Βάσεων Δεδομένων

Οι βάσεις δεδομένων κατηγοριοποιούνται ανάλογα με το μοντέλο δεδομένων που χρησιμοποιούν. Οι **σχεσιακές βάσεις δεδομένων (Relational Databases)**, όπως η MySQL, η PostgreSQL και η Oracle DB, βασίζονται σε πίνακες με γραμμές και στήλες, όπου οι σχέσεις μεταξύ τους ορίζονται μέσω κλειδιών [16]. Παλαιότερα μοντέλα, όπως οι **ιεραρχικές** και **δικτυωτές βάσεις**, έχουν περιοριστεί σε συγκεκριμένες εφαρμογές λόγω έλλειψης ευελιξίας. Οι **αντικειμενοστραφείς βάσεις** χρησιμοποιούνται σε περιβάλλοντα που απαιτούν αντιστοίχιση δεδομένων με αντικείμενα προγραμματισμού.

Οι **NoSQL βάσεις** έχουν αναδειχθεί ως η κύρια εναλλακτική για εφαρμογές με μη σχεσιακά, ευέλικτα ή μεγάλης κλίμακας δεδομένα. Διακρίνονται σε υποκατηγορίες:

- **Βάσεις εγγράφων (Document-based):** Αποθηκεύουν δεδομένα σε μορφή JSON ή XML (π.χ., MongoDB).
- **Βάσεις κλειδιού-τιμής (Key-Value):** Βελτιστοποιημένες για γρήγορη ανάκτηση (π.χ., Redis).
- **Βάσεις στηλών (Column-family):** Κατάλληλες για ανάλυση μεγάλων συνόλων δεδομένων (π.χ., Cassandra).
- **Βάσεις γραφημάτων (Graph):** Χρησιμοποιούνται για αναπαράσταση σύνθετων σχέσεων (π.χ., Neo4j) [18].

SQL vs NoSQL

Η **SQL (Structured Query Language)** είναι η κλασική γλώσσα για διαχείριση σχεσιακών βάσεων, προσφέροντας δυνατότητες όπως δημιουργία πινάκων, εισαγωγή δεδομένων και εκτέλεση πολύπλοκων ερωτημάτων. Ωστόσο, στις σύγχρονες εφαρμογές με ημιδομημένα δεδομένα ή απαιτήσεις υψηλής κλιμάκωσης, οι **NoSQL βάσεις** κερδίζουν έδαφος λόγω ευελιξίας, ταχύτητας και ικανότητας να λειτουργούν σε κατανεμημένα περιβάλλοντα [19].

- **Firebase: Μια Σύγχρονη NoSQL Λύση**

Η **Firebase**, μια πλατφόρμα **Backend-as-a-Service (BaaS)** που αναπτύχθηκε από την Google, προσφέρει εργαλεία για ανάπτυξη εφαρμογών χωρίς την ανάγκη για προσαρμοσμένο back-end. Βασικό της στοιχείο είναι οι NoSQL βάσεις δεδομένων **Firebase Realtime Database** και **Cloud Firestore**, οι οποίες χρησιμοποιούν τη μορφή JSON για αποθήκευση και ανταλλαγή δεδομένων [20].

Πλεονεκτήματα της Firebase
Η Firebase ξεχωρίζει για τον **πραγματικό χρόνο συγχρονισμού**, όπου οι αλλαγές στα δεδομένα αντικατοπτρίζονται άμεσα σε όλες τις συνδεδεμένες συσκευές χωρίς την ανάγκη για επιπλέον αιτήματα [21]. Επίσης, η **ευκολία ενσωμάτωσης** υποστηρίζεται από SDKs για πλατφόρμες όπως Android, iOS, Web και Unity, απλοποιώντας τη δημιουργία mobile εφαρμογών [22]. Η **επεκτασιμότητά** της, με υποστήριξη από την Google Cloud, επιτρέπει την ανάπτυξη από πρωτότυπα (MVPs) έως εφαρμογές με εκατομμύρια χρήστες.

Η πλατφόρμα ενσωματώνει **κανόνες ασφαλείας** για έλεγχο πρόσβασης σε επίπεδο εγγράφου ή πεδίου, ενώ η **serverless αρχιτεκτονική** της απαλλάσσει τους προγραμματιστές από τη διαχείριση servers, μειώνοντας το λειτουργικό κόστος [22]. Επιπλέον, η ενοποίηση με υπηρεσίες όπως **Firebase Authentication**, **Cloud Messaging** και **Machine Learning APIs** προσφέρει ένα ολοκληρωμένο οικοσύστημα ανάπτυξης [20].

Σύγκριση με Παραδοσιακές SQL Βάσεις

Σε αντίθεση με τις σχεσιακές βάσεις όπως η MySQL, η Firebase δεν απαιτεί **προκαθορισμένο σχήμα**, προσφέροντας ευελιξία στην αποθήκευση

ημιδομημένων δεδομένων. Ενώ οι SQL βάσεις βασίζονται σε πίνακες και σχέσεις, η Firebase χρησιμοποιεί δομές εγγράφων JSON, ιδανικές για δυναμικές εφαρμογές. Ο **πραγματικός χρόνος συγχρονισμού** είναι ένα κύριο πλεονέκτημα έναντι των παραδοσιακών λύσεων, που συχνά απαιτούν επιπλέον ρυθμίσεις για παρόμοια λειτουργικότητα.

Ωστόσο, οι SQL βάσεις διατηρούν πλεονεκτήματα σε εφαρμογές με **πολύπλοκα ερωτήματα** ή αυστηρές απαιτήσεις συνοχής δεδομένων (ACID compliance). Η Firebase, από την άλλη, είναι βελτιστοποιημένη για **ταχύτητα ανάπτυξης** και εφαρμογές που απαιτούν άμεση αλληλεπίδραση, όπως κοινωνικά δίκτυα ή συστήματα συναγερμών [21].

Συνολικά, η Firebase αποτελεί μια ιδανική επιλογή για **mobile και web εφαρμογές** όπου η ταχύτητα, η ευκολία και ο πραγματικός χρόνος είναι κρίσιμοι, ενώ οι παραδοσιακές SQL λύσεις εξακολουθούν να κυριαρχούν σε εφαρμογές με δομημένα δεδομένα και περίπλοκη λογική ερωτημάτων [22].

2.2 Τεχνολογίες IoT

Το Διαδίκτυο των Πραγμάτων (Internet of Things, IoT) αναφέρεται στο δίκτυο φυσικών συσκευών (π.χ., αισθητήρων, actuators, έξυπνων συσκευών) που συνδέονται μέσω του διαδικτύου για τη συλλογή, ανταλλαγή και επεξεργασία δεδομένων σε πραγματικό χρόνο. Η τεχνολογία IoT βασίζεται σε **ενσωματωμένα συστήματα, ασύρματα πρωτόκολλα επικοινωνίας** (π.χ., MQTT, LoRaWAN, 5G) και **πλατφόρμες cloud** (π.χ., AWS IoT, Azure IoT) για την ολοκληρωμένη διαχείριση δεδομένων [31]. Κεντρική προϋπόθεση για τη λειτουργία του IoT είναι η δυνατότητα **σύνδεσης χαμηλής κατανάλωσης ενέργειας** (Low-Power Wide-Area Networks - LPWAN), η οποία εξασφαλίζει μακρά διάρκεια ζωής σε συσκευές με περιορισμένη μπαταρία [32].

Οι εφαρμογές του IoT καλύπτουν τομείς όπως οι **έξυπνες πόλεις** (smart cities), όπου αισθητήρες παρακολουθούν την ποιότητα αέρα ή τη διαχείριση κυκλοφορίας, και η **βιομηχανία 4.0**, με αυτοματοποιημένα συστήματα παρακολούθησης παραγωγικών διαδικασιών [33]. Η ανάπτυξη IoT απαιτεί την εφαρμογή **τεχνολογιών edge computing**, οι οποίες μειώνουν την καθυστέρηση με την επεξεργασία δεδομένων κοντά στην πηγή τους, αντί να βασίζονται αποκλειστικά στο cloud [34]. Παράλληλα, η ένταξη τεχνητής νοημοσύνης (AI) και μηχανικής μάθησης (ML) επιτρέπει την ανάλυση μεγάλων συνόλων δεδομένων (big data) για πρόβλεψη βλαβών ή βελτιστοποίηση λειτουργιών [35].

Οι κύριες προκλήσεις του IoT περιλαμβάνουν ζητήματα **ασφάλειας και ιδιωτικότητας**, λόγω της ευαισθησίας των δεδομένων και του κινδύνου από cyberεπιθέσεις σε ασυνήθιστα ασφαλή συσκευές [36]. Επιπλέον, η **ετερογένεια των πρωτοκόλλων** και η έλλειψη καθολικών προτύπων δυσχεραίνουν τη διαλειτουργικότητα συσκευών από διαφορετικούς κατασκευαστές [37]. Παρά αυτά, το IoT συνεχίζει να επεκτείνεται, με προβλέψεις να φτάνει τα **75 δισεκατομμύρια συνδεδεμένες συσκευές παγκοσμίως έως το 2025** [38].

2.3 Πρωτόκολλα Επικοινωνίας

Τα πρωτόκολλα επικοινωνίας αποτελούν το θεμέλιο της διασύνδεσης συσκευών και συστημάτων, καθορίζοντας τους κανόνες και τις διαδικασίες για την ανταλλαγή δεδομένων. Η επιλογή του κατάλληλου πρωτοκόλλου εξαρτάται από παράγοντες όπως ο τομέας εφαρμογής (βιομηχανία, IoT, web), οι τεχνικοί περιορισμοί (καθυστέρηση, εύρος ζώνης, ενέργεια) και οι απαιτήσεις ασφάλειας. Στα σύγχρονα συστήματα, η συνύπαρξη πολλαπλών πρωτοκόλλων απαιτεί ευελιξία και διαλειτουργικότητα, ιδιαίτερα σε ολοκληρωμένες λύσεις που συνδυάζουν υλικό, λογισμικό και cloud υπηρεσίες [39].

Γενική Σύνοψη Πρωτοκόλλων

Στον τομέα της **βιομηχανίας**, πρωτόκολλα όπως το **OPC UA** και το **Modbus** επικεντρώνονται σε αξιοπιστία, επικοινωνία σε πραγματικό χρόνο και συμβατότητα με παλαιότερες υποδομές. Για εφαρμογές **IoT**, πρωτόκολλα όπως το **MQTT** βελτιστοποιούνται για χαμηλή κατανάλωση ενέργειας και ασύρματη επικοινωνία, ενώ τα **web πρωτόκολλα** (π.χ., REST API, HTTP) διασφαλίζουν κλιμακωσιμότητα και ενσωμάτωση με υπηρεσίες τρίτων. Σε πρωτόγονα συστήματα ή πρωτότυπα, η **σειριακή επικοινωνία** (Serial) παραμένει μια απλή και αξιόπιστη λύση.

Λεπτομέρειες για Κάθε Πρωτόκολλο

OPC-UA(UnifiedArchitecture)

Το OPC UA είναι ένα **client-server πρωτόκολλο** για βιομηχανικά συστήματα, που βασίζεται σε ένα node-based μοντέλο. Κάθε συσκευή, αισθητήρας ή διεργασία αντιστοιχεί σε έναν κόμβο (node), επιτρέποντας την αναπαράσταση πολύπλοκων δομών δεδομένων. Το πρωτόκολλο υποστηρίζει ενσωματωμένη **κρυπτογράφηση** και αυθεντικοποίηση, καθιστώντας το ιδανικό για εφαρμογές όπως συστήματα ελέγχου (PLCs) ή SCADA, όπου η ασφάλεια και η διαλειτουργικότητα είναι κρίσιμες [40].

Modbus-TCP

Το Modbus TCP είναι ένα **register-based πρωτόκολλο** που λειτουργεί πάνω σε Ethernet. Κάθε συσκευή αναγνωρίζεται μέσω διευθύνσεων μνήμης (π.χ., holding registers για αναλογικές τιμές). Χρησιμοποιείται ευρέως σε βιομηχανικά συστήματα για απλές ανταλλαγές δεδομένων μεταξύ PLCs και SCADA. Ωστόσο, η έλλειψη ενσωματωμένων μηχανισμών ασφαλείας, όπως κρυπτογράφηση, περιορίζει τη χρήση του σε περιβάλλοντα με αυστηρές απαιτήσεις [41].

MQTT&TheThingsNetwork(TTN)

Το MQTT είναι ένα **publish-subscribe πρωτόκολλο** βελτιστοποιημένο για IoT, με έμφαση στη χαμηλή κατανάλωση ενέργειας. Συνδυάζεται συχνά με το **The Things Network (TTN)**, μια πλατφόρμα που δρομολογεί μηνύματα από συσκευές LoRaWAN σε MQTT brokers (π.χ., cloud servers). Αυτή η αρχιτεκτονική είναι ιδανική για εφαρμογές όπως γεωργικοί αισθητήρες, όπου απαιτείται μετάδοση δεδομένων σε μεγάλη εμβέλεια με ελάχιστη ενέργεια [42].

SerialCommunication

Η **σειριακή επικοινωνία** (π.χ., UART, RS-232) αποτελεί τη βασική μορφή επικοινωνίας μεταξύ μικροελεγκτών (Arduino) και υπολογιστών. Χρησιμοποιείται σε πρωτότυπα ή συστήματα με απλές απαιτήσεις, όπως η ανάγνωση δεδομένων από αισθητήρες θερμοκρασίας. Παρά την απλότητά της, η έλλειψη κλιμακωσιμότητας και ασφαλείας την περιορίζει σε μικρής κλίμακας εφαρμογές [43].

RESTAPI

Η **REST API** είναι μια δημοφιλής αρχιτεκτονική για επικοινωνία frontend-backend μέσω HTTP μεθόδων (GET, POST, κ.ά.). Τα δεδομένα ανταλλάσσονται σε μορφές όπως JSON, επιτρέποντας εύκολη ενσωμάτωση με εφαρμογές React ή Angular. Χαρακτηριστικά της είναι οι **stateless operations** και η χρήση endpoints (π.χ., /api/data), που την καθιστούν ιδανική για web εφαρμογές με υψηλές απαιτήσεις κλιμακωσιμότητας [44].

HTTPProxy

Ο **HTTP Proxy** λειτουργεί ως ενδιάμεσος διακομιστής που διαχειρίζεται αιτήματα προς εξωτερικά APIs (π.χ., WeatherAPI). Παρέχει ασφάλεια μέσω απόκρυψης API κλειδίων, caching για μείωση φόρτου και διαχείριση CORS (Cross-Origin Resource Sharing). Είναι απαραίτητος σε περιπτώσεις όπου το frontend δεν μπορεί να απευθύνεται απευθείας σε τρίτους παρόχους, διασφαλίζοντας παράλληλα την προστασία δεδομένων [45].

Συνοπτική-Αξιολόγηση

Κάθε πρωτόκολλο εξυπηρετεί συγκεκριμένες ανάγκες:

OPC UA και **Modbus** κυριαρχούν σε βιομηχανικά συστήματα λόγω αξιοπιστίας.

MQTT και **TTN** αποτελούν την καρδιά των IoT εφαρμογών με χαμηλή ενέργεια.

REST API και **HTTP Proxy** διασφαλίζουν την ενσωμάτωση web υπηρεσιών και cloud.

Η **Serial Communication** παραμένει αναγκαία για πρωτότυπα και απλές ροές δεδομένων.

Η επιλογή του κατάλληλου πρωτοκόλλου εξαρτάται από την ισορροπία μεταξύ τεχνικών απαιτήσεων, ασφάλειας και κλιμακωσιμότητας, με πολλά συστήματα να συνδυάζουν πολλαπλά πρωτόκολλα για βελτιστοποίηση [39].

2.4 Εφαρμογές Παρακολούθησης στη Βιομηχανία

Οι εφαρμογές παρακολούθησης στη βιομηχανία έχουν μετασχηματίσει τη διαχείριση παραγωγικών διεργασιών μέσω της συλλογής και ανάλυσης δεδομένων σε πραγματικό χρόνο. Βασικό εργαλείο αποτελούν τα **Συστήματα SCADA (Supervisory Control and Data Acquisition)**, τα οποία ενσωματώνουν αισθητήρες, PLCs και γραφικές διεπαφές για την παρακολούθηση παραμέτρων όπως θερμοκρασία, πίεση και κατανάλωση ενέργειας. Αυτά τα συστήματα επιτρέπουν τη λήψη τακτικών αποφάσεων και την αυτόματη ενεργοποίηση συναγερμών σε περίπτωση αποκλίσεων [46].

Η έλευση του **Industry 4.0** έχει ενισχύσει τη χρήση **Ψηφιακών Διδύμων (Digital Twins)**, δηλαδή εικονικών αντιγράφων φυσικών συστημάτων, που προσομοιώνουν τη συμπεριφορά μηχανημάτων ή γραμμών παραγωγής. Οι ψηφιακοί δίδυμοι επιτρέπουν την πρόβλεψη βλαβών, τη βελτιστοποίηση παραγωγής και την εκπαίδευση προσωπικού χωρίς διακοπές λειτουργίας [47]. Παράλληλα, η **Προγνωστική Συντήρηση (Predictive Maintenance)**, με τη βοήθεια τεχνητής νοημοσύνης και machine learning, αναλύει ιστορικά δεδομένα και προβλέπει πιθανές αστοχίες πριν από την εμφάνισή τους, μειώνοντας το κόστος διακοπών και επεκτείνοντας τον κύκλο ζωής εξοπλισμού [48].

Στον τομέα της **Ποιότητας (Quality Control)**, συστήματα παρακολούθησης βασισμένα σε **υπολογιστική όραση (computer vision)** και αισθητήρες υπερέθρων ελέγχουν αυτόματα τη συμμόρφωση προϊόντων με προδιαγραφές, εντοπίζοντας ελαττώματα με ακρίβεια που ξεπερνά την ανθρώπινη [49]. Επιπλέον, πλατφόρμες όπως το **Azure IoT** ή το **Siemens MindSphere** προσφέρουν ολοκληρωμένες λύσεις για τη συγκέντρωση δεδομένων από πολλαπλές πηγές (PLC, IoT συσκευές, ERP συστήματα) και την απεικόνισή τους σε dashboards, διευκολύνοντας την κατανόηση πολύπλοκων βιομηχανικών διαδικασιών [50].

Οι κύριες προκλήσεις περιλαμβάνουν την **ασφάλεια δεδομένων** (κίνδυνοι από cyberεπιθέσεις σε συνδεδεμένα συστήματα) και την **ετερογένεια των πρωτοκόλλων επικοινωνίας**, η οποία δυσχεραίνει την ομαλή ενοποίηση συσκευών διαφορετικών κατασκευαστών [51]. Παρά αυτά, η αγορά βιομηχανικής παρακολούθησης αναμένεται να φτάσει **\$45 δισεκατομμύρια έως το 2027**, με κύριους παράγοντες την υιοθέτηση IoT και τη βελτίωση των αλγορίθμων AI [52].

3 Αρχιτεκτονική Συστήματος

3.1 Ανάλυση Απαιτήσεων και Τεχνικών Προδιαγραφών

Η ανάπτυξη της εφαρμογής βασίστηκε στην ανάγκη για ένα σύστημα παρακολούθησης δεδομένων από βιομηχανικούς αισθητήρες, το οποίο θα είναι φιλικό προς τον χρήστη, επεκτάσιμο και τεχνικά αξιόπιστο. Η ανάλυση απαιτήσεων περιλαμβάνει τόσο λειτουργικές όσο και μη λειτουργικές παραμέτρους. Οι βασικές λειτουργικές απαιτήσεις περιλαμβάνουν την αυθεντικοποίηση χρηστών, την εγγραφή και παρακολούθηση αισθητήρων, την απεικόνιση δεδομένων σε γραφήματα και μετρητές σε πραγματικό χρόνο, την προβολή γεωγραφικών θέσεων μέσω διαδραστικού χάρτη και τη δυνατότητα εξαγωγής δεδομένων σε αρχεία CSV.

Μη λειτουργικές απαιτήσεις περιλαμβάνουν την ασφάλεια των προσωπικών δεδομένων, την απόδοση του συστήματος υπό συνεχή ροή δεδομένων, τη σταθερότητα κατά την αποστολή και λήψη από πολλαπλούς αισθητήρες και την ευχρηστία της διεπαφής.

Η υλοποίηση προβλέπει την ύπαρξη δύο βασικών επιπέδων: το frontend, το οποίο αναλαμβάνει την παρουσίαση, και το backend, το οποίο διαχειρίζεται την επιχειρησιακή λογική, την αποθήκευση και τη διασύνδεση με τις πηγές δεδομένων. Οι χρήστες έπρεπε να μπορούν να διαχειρίζονται αισθητήρες οι οποίοι λειτουργούν μέσω διαφορετικών τεχνολογιών (Arduino, PLCs, LoRaWAN/TTN, APIs) και να προβάλλουν τις μετρήσεις σε πραγματικό χρόνο, μέσω γραφημάτων, gauge μετρητών και δυναμικής απεικόνισης σε χάρτη.

Κρίσιμη προδιαγραφή ήταν επίσης η δυνατότητα επέκτασης του συστήματος, τόσο σε επίπεδο νέων χρηστών όσο και σε επίπεδο εισαγωγής νέων τύπων αισθητήρων και δεδομένων. Η αρχιτεκτονική της εφαρμογής έπρεπε να είναι loosely coupled, ώστε κάθε υποσύστημα να μπορεί να αναπτυχθεί ή να συντηρηθεί αυτόνομα, χωρίς επιπτώσεις στη συνολική λειτουργία. Για τον σκοπό αυτό, προτιμήθηκε η χρήση RESTful APIs για την επικοινωνία μεταξύ των επιπέδων της εφαρμογής και η υλοποίηση των βασικών λειτουργιών σε ασύγχρονο περιβάλλον (JavaScript/Node.js) [1].

Η ασφάλεια προσεγγίστηκε με βάση τις συστάσεις του OWASP για web εφαρμογές. Προβλέφθηκε η χρήση hashing για την αποθήκευση κωδικών, ο διαχωρισμός frontend και backend με χρήση CORS, καθώς και η απομόνωση των διαπιστευτηρίων σε περιβαλλοντικές μεταβλητές μέσω αρχείων `.env` [2]. Οι προδιαγραφές επίσης απαιτούσαν δυνατότητα real-time λειτουργίας, με υποστήριξη για live updates χωρίς επαναφόρτωση σελίδας, μέσω hooks στο frontend και μηχανισμών polling ή subscriptions στο backend.

Η τελική σύνοψη των απαιτήσεων συγκεντρώνει: (α) πολλαπλές πηγές δεδομένων, (β) πραγματικού χρόνου απεικόνιση, (γ) user authentication, (δ) δυνατότητα εξαγωγής αρχείων και (ε) πλήρως παραμετροποιήσιμη διεπαφή. Η τεχνική ανάλυση

αυτών των προδιαγραφών αποτέλεσε το θεμέλιο για την επιλογή τεχνολογιών που ακολουθεί στο επόμενο υποκεφάλαιο.

3.2 Συνολική Αρχιτεκτονική Συστήματος

Η συνολική αρχιτεκτονική της εφαρμογής ακολουθεί την αρχή του διαχωρισμού ανησυχιών (separation of concerns) και βασίζεται σε client-server μοντέλο, όπου το frontend και το backend αποτελούν διακριτές οντότητες που επικοινωνούν μέσω RESTful APIs. Η προσέγγιση αυτή επιτρέπει την ανεξάρτητη ανάπτυξη, δοκιμή και συντήρηση των δύο πλευρών του συστήματος, διατηρώντας παράλληλα την ευελιξία για μελλοντική επεκτασιμότητα [3].

Το frontend έχει αναπτυχθεί με React.js και λειτουργεί ως Single Page Application (SPA), προσφέροντας μια δυναμική και γρήγορη εμπειρία χρήσης. Ο χρήστης συνδέεται στην εφαρμογή μέσω login/register μηχανισμών και αποκτά πρόσβαση στο περιβάλλον παρακολούθησης αισθητήρων, όπου μπορεί να δει δεδομένα σε γραφήματα, χάρτες και μετρητές. Η λογική του UI διαχειρίζεται την κατάσταση του χρήστη, τα tokens εισόδου, και επικοινωνεί με το backend μέσω ασφαλών HTTP αιτημάτων για την ανάκτηση ή αποστολή δεδομένων.

Το backend είναι υλοποιημένο με Node.js και Express, παρέχοντας όλα τα REST endpoints για authentication, διαχείριση αισθητήρων, αποθήκευση μετρήσεων, σύνδεση με εξωτερικές πηγές και εξαγωγή δεδομένων. Εσωτερικά οργανώνεται σε modules για routes, controllers, middleware και helpers. Η επικοινωνία με τις βάσεις δεδομένων γίνεται μέσω των βιβλιοθηκών mysql2 για τη MySQL και firebase-admin για τη Firebase Realtime Database.

Η εφαρμογή υποστηρίζει τέσσερις βασικούς τύπους αισθητήρων:

1. **Arduino** (μέσω USB σειριακής σύνδεσης)
2. **PLCs** (μέσω OPC UA)
3. **LoRaWAN μέσω The Things Network** (με MQTT subscriptions)
4. **Εξωτερικά APIs** (όπως Weather API)

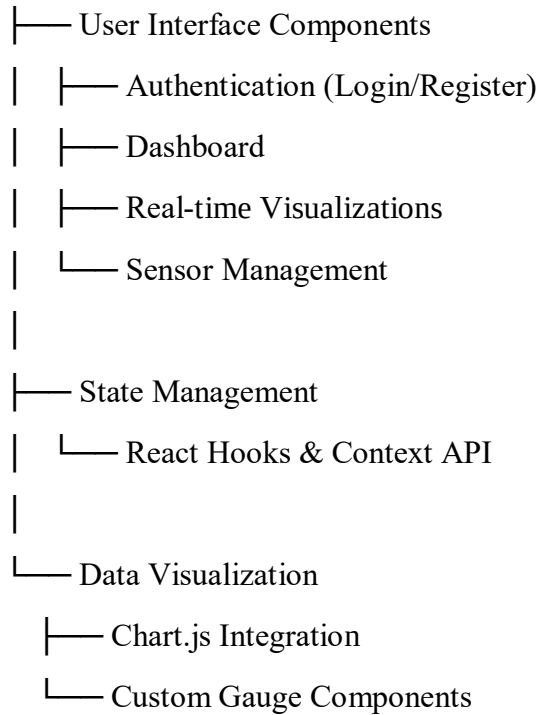
Κάθε αισθητήρας καταγράφεται από τον χρήστη μέσω του dashboard και διαχειρίζεται από ειδικό handler module στο backend, το οποίο λειτουργεί ως bridge μεταξύ του πρωτοκόλλου επικοινωνίας του αισθητήρα και του αποθηκευτικού μοντέλου της εφαρμογής. Οι μετρήσεις αποθηκεύονται στο Firebase, δομημένες ανά χρήστη και αισθητήρα, και παρέχονται σε πραγματικό χρόνο στο frontend για άμεση απεικόνιση.

Η συνολική αρχιτεκτονική υποστηρίζεται από μηχανισμούς ασφαλείας (όπως hashing, session handling και έλεγχο πρόσβασης), από responsive σχεδιασμό στο frontend και από πλήρη αποκεντρωση δεδομένων ανά χρήστη, επιτρέποντας ταυτόχρονη χρήση του συστήματος από πολλαπλούς ανεξάρτητους χρήστες. Επιπλέον, έχει προβλεφθεί η υποδομή για επέκταση με modules Τεχνητής Νοημοσύνης ή Rule-based συστημάτων που θα εκτελούνται είτε τοπικά είτε ως ξεχωριστά microservices.

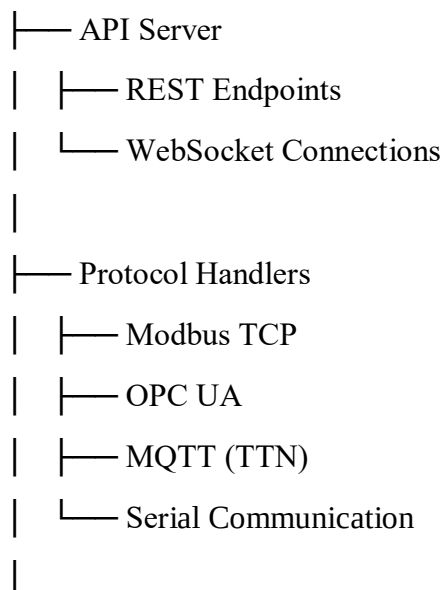
Η αρχιτεκτονική αυτή εξασφαλίζει ευελιξία, αξιοπιστία και υψηλή απόδοση, ενώ είναι συμβατή με τις αρχές σχεδίασης λογισμικού για εφαρμογές IoT μεγάλης κλίμακας [4].

System Architecture

1. Frontend Layer (React.js)



2. Backend Layer (Node.js/Express)



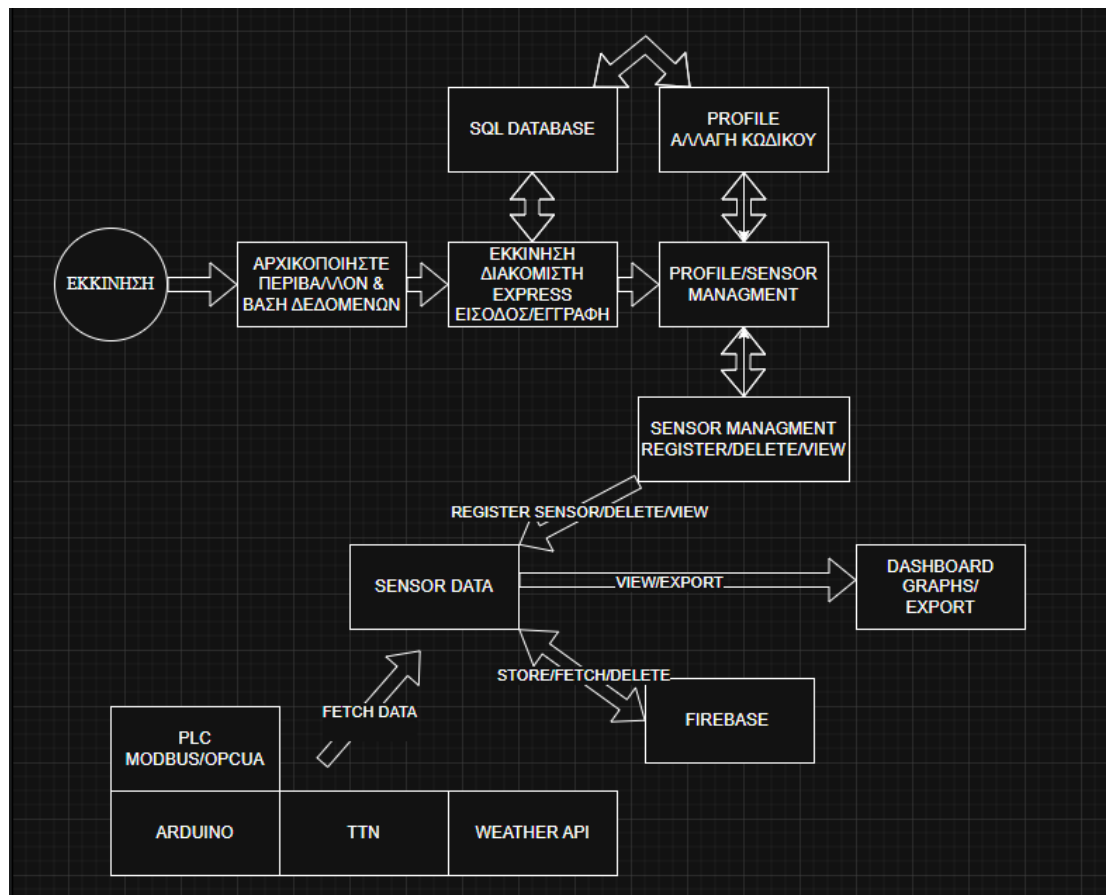
- └─ Data Processing
 - └─ Real-time Analytics
 - └─ Threshold Monitoring

3. Data Layer

- └─ Firebase Realtime Database
 - └─ Sensor Configurations
 - └─ Time-series Data
- └─ MySQL Database
 - └─ User Authentication

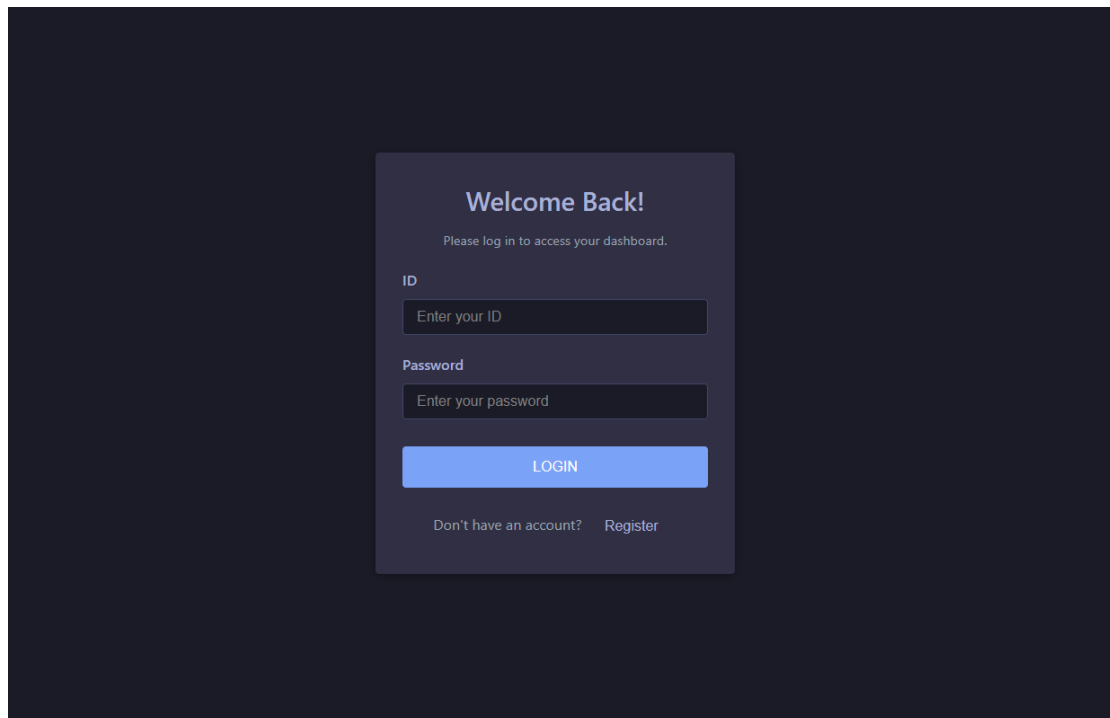
3.3 Διάγραμμα Ροής και Αλληλεπίδρασης

Η λειτουργία της εφαρμογής βασίζεται σε μια οργανωμένη ροή ενεργειών μεταξύ χρήστη, frontend, backend και εξωτερικών πηγών δεδομένων, με σκοπό τη συλλογή, απεικόνιση και αποθήκευση μετρήσεων από αισθητήρες. Η αλληλεπίδραση αυτών των οντοτήτων μπορεί να περιγραφεί μέσω διαγράμματος ροής που περιλαμβάνει διακριτά στάδια: αυθεντικοποίηση χρήστη, επιλογή αισθητήρα, επικοινωνία με backend, άντληση ή λήψη δεδομένων και τελική οπτικοποίηση στο περιβάλλον του dashboard.



Εικόνα 1: Διάγραμμα ροής λειτουργίας και αλληλεπίδρασης του συστήματος

Η διαδικασία ξεκινάει όταν ο χρήστης επισκέπτεται την εφαρμογή και είτε πραγματοποιεί εγγραφή είτε σύνδεση μέσω της φόρμας login. Το frontend αποστέλλει το αίτημα εισόδου στο backend, όπου ελέγχεται η εγκυρότητα των διαπιστευτηρίων μέσω σύγκρισης του καταχωρημένου email και του bcrypt-hashed κωδικού με τα δεδομένα της βάσης MySQL. Εφόσον η είσοδος είναι επιτυχής, αποστέλλεται επιβεβαίωση στο frontend, ενεργοποιώντας την πρόσβαση στο dashboard.

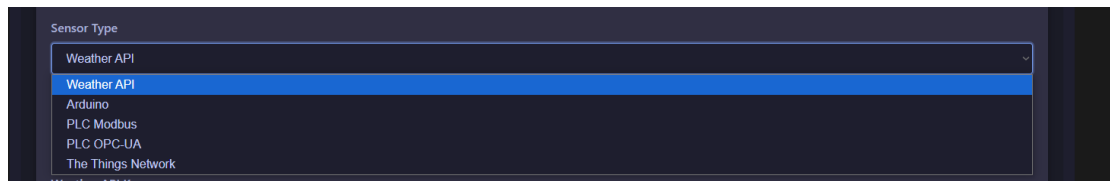


Εικόνα 2: Διεπαφή χρήστη με βασικές επιλογές στο αρχικό μενού

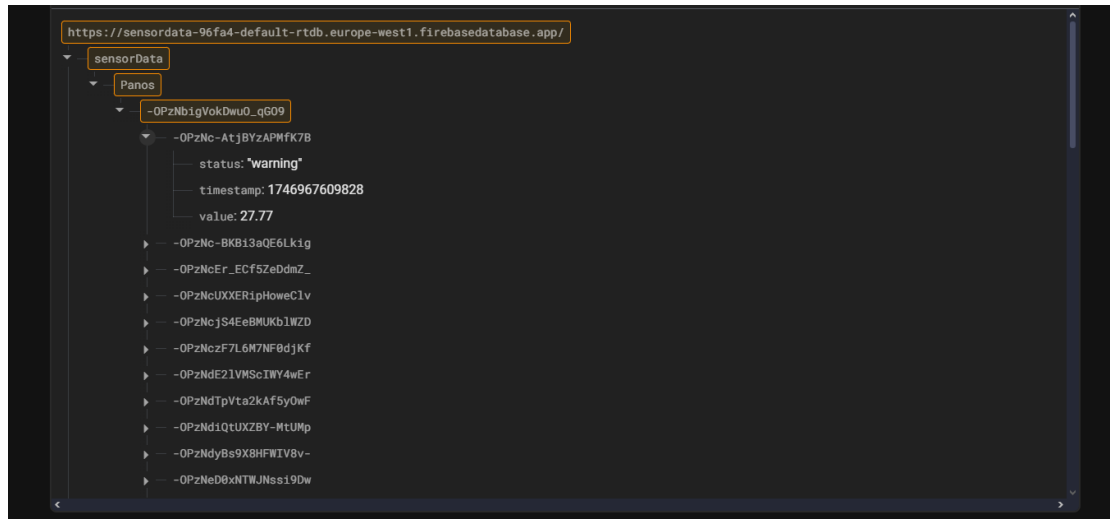
Αμέσως μετά, ο χρήστης μπορεί να προσθέσει ή να επιλέξει έναν από τους καταχωρημένους αισθητήρες του. Κατά την επιλογή, το frontend στέλνει σχετικό αίτημα στο backend για τη λήψη δεδομένων, με βάση το sensorId. Το backend, ανάλογα με τον τύπο του αισθητήρα, καλεί τον κατάλληλο handler: για Arduino χρησιμοποιεί σειριακή ανάγνωση, για PLCs αποστολή αιτήματος μέσω Modbus, για LoRaWAN εγγραφή σε MQTT topic, ενώ για APIs χρησιμοποιείται HTTP client (Axios) για λήψη των δεδομένων. Όλα τα δεδομένα αποθηκεύονται ή ενημερώνονται στη Firebase σε πραγματικό χρόνο, με ταξινόμηση ανά χρήστη και αισθητήρα.



Εικόνα 3: Διεπαφή χρήστη με Register sensor

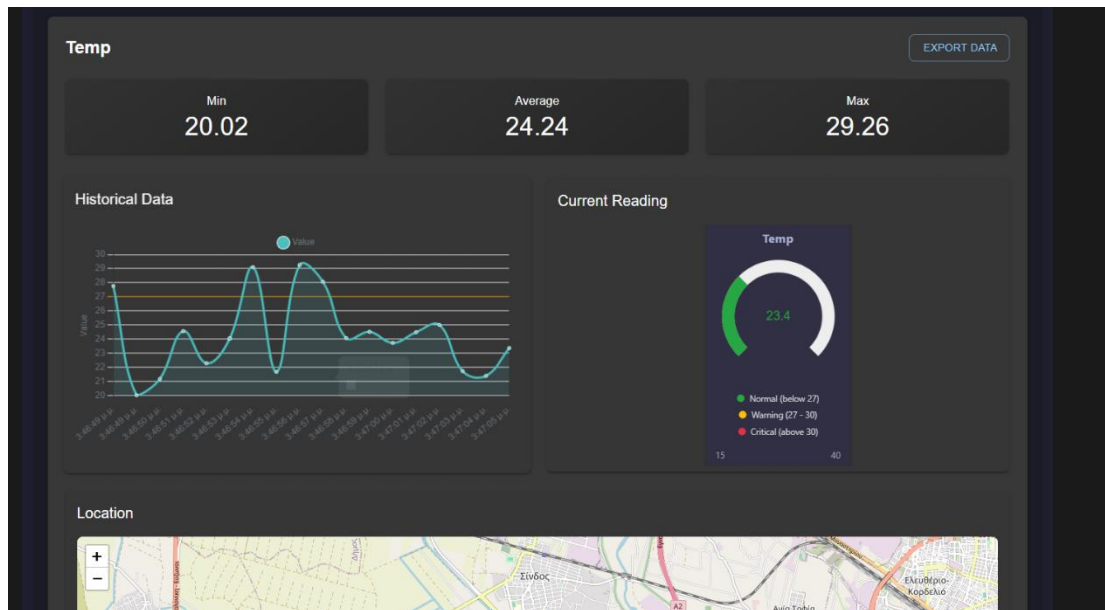


Εικόνα 4: Επιλογές Sensor



Εικόνα 5: Data στο Firebase

Ταυτόχρονα, το frontend λαμβάνει είτε τα τελευταία ιστορικά δεδομένα είτε ροή σε ζωντανό χρόνο, τα οποία φορτώνονται στο UI μέσω chart components, gauge meters και διαδραστικό χάρτη. Κάθε νέα μέτρηση αποτυπώνεται οπτικά, ενώ αν ξεπεραστούν προκαθορισμένα όρια, εμφανίζονται alert ενδείξεις. Ο χρήστης μπορεί ανά πάσα στιγμή να εξάγει τα δεδομένα του αισθητήρα σε αρχείο CSV, ενεργοποιώντας client-side συνάρτηση που μετατρέπει τα JSON δεδομένα σε μορφή υπολογιστικού φύλλου και τα κατεβάζει τοπικά.



Εικόνα 6: Dashboard View Sensor

Ολοκληρώνοντας τη χρήση, ο χρήστης μπορεί να αποσυνδεθεί, διαγράφοντας το session του από το frontend και ακυρώνοντας τη σύνδεση με το backend. Η ροή αυτή διασφαλίζει την ανεμπόδιστη λειτουργία της εφαρμογής, επιτρέποντας την ταυτόχρονη διαχείριση χρηστών και αισθητήρων με απόλυτο έλεγχο δεδομένων και άμεση απόκριση, είτε σε στατικό είτε σε real-time περιβάλλον.

Η αλληλεπίδραση μεταξύ των επιπέδων αυτών υλοποιείται με βάση αρχές RESTful σχεδίασης και παρακολουθείται από middleware ελέγχους, προσφέροντας ασφαλές, modular και επεκτάσιμο μοντέλο ροής δεδομένων σε ένα σύστημα IoT πολλαπλών εισόδων και λειτουργιών [5].

3.4 Επιλογή Τεχνολογιών και Βιβλιοθηκών

Η επιλογή των τεχνολογιών που χρησιμοποιήθηκαν στην παρούσα εργασία βασίστηκε σε κριτήρια όπως η απόδοση, η ευχρηστία, η υποστήριξη κοινότητας, η επεκτασιμότητα και η δυνατότητα ενσωμάτωσης με διαφορετικά πρωτόκολλα αισθητήρων. Έγινε συνειδητή προσπάθεια ώστε το τεχνολογικό υπόβαθρο της εφαρμογής να είναι σύγχρονο, modular και προσαρμοσμένο στις ανάγκες μιας ευέλικτης βιομηχανικής IoT πλατφόρμας.

Στο frontend επιλέχθηκε το React.js λόγω της component-based αρχιτεκτονικής του, η οποία επιτρέπει την κατασκευή επαναχρησιμοποιήσιμων και ανεξάρτητων UI στοιχείων. Επιπλέον, η μεγάλη κοινότητα και η πληθώρα βιβλιοθηκών που υποστηρίζουν το React (όπως το React Router για πλοήγηση και το Axios για

αιτήματα HTTP) το καθιστούν ιδανικό εργαλείο για γρήγορη και αξιόπιστη ανάπτυξη Single Page Applications (SPAs) [6]. Παράλληλα, η χρήση του Material-UI (MUI) επέτρεψε την ενσωμάτωση έτοιμων responsive UI components, προσφέροντας αισθητικά και λειτουργικά αποτελέσματα με λιγότερο custom CSS.

Για την οπτικοποίηση των δεδομένων αισθητήρων χρησιμοποιήθηκαν δύο βασικές βιβλιοθήκες: το Chart.js μέσω του `react-chartjs-2` και το Recharts. Το Chart.js επιλέχθηκε για τη δυνατότητα απόδοσης σύνθετων και ακριβών γραφημάτων, ενώ το Recharts προσφέρει βελτιωμένη υποστήριξη για responsive περιβάλλον και καλύτερη ενσωμάτωση με React. Η επιλογή αυτών των δύο συνδυαστικά επέτρεψε την υλοποίηση στατικών και real-time γραφημάτων για τις μετρήσεις των αισθητήρων, με δυνατότητα επεξεργασίας σε UI επίπεδο [7].

Το backend της εφαρμογής βασίστηκε σε Node.js και Express, καθώς προσφέρουν γρήγορη, μη-μπλοκαριστική εκτέλεση εντολών (non-blocking I/O), και είναι ιδανικά για real-time εφαρμογές με πολλαπλές ταυτόχρονες συνδέσεις. Το Express προσφέρει απλό routing, middleware αρχιτεκτονική και γρήγορη δημιουργία RESTful APIs. Επιπλέον, υποστηρίζεται από πλήθος πακέτων npm, επιτρέποντας εύκολη σύνδεση με βάσεις δεδομένων, εξωτερικές υπηρεσίες και πρωτόκολλα συσκευών [8].

Για την αποθήκευση των δεδομένων χρηστών χρησιμοποιήθηκε MySQL, μια αξιόπιστη σχεσιακή βάση δεδομένων, κατάλληλη για εγγραφές με σταθερή δομή, όπως credentials και metadata. Οι μετρήσεις των αισθητήρων αποθηκεύονται στη Firebase Realtime Database, η οποία επελέγη λόγω της δυνατότητάς της για real-time συγχρονισμό δεδομένων, αποθήκευση σε μορφή JSON και εύκολη ενσωμάτωση με JavaScript εφαρμογές [9].

Στην πλευρά της διασύνδεσης με αισθητήρες, χρησιμοποιήθηκαν επιμέρους βιβλιοθήκες ανά τύπο: `serialport` για συσκευές Arduino μέσω USB, `modbus-serial` για PLCs, `node-opcua` για επικοινωνία με OPC UA servers και `mqtt` για λήψη πακέτων δεδομένων από το The Things Network. Όλες οι βιβλιοθήκες προσφέρουν ασφαλείς και τεκμηριωμένες διεπαφές για επικοινωνία με φυσικές ή εικονικές συσκευές, και υποστηρίζουν callbacks, reconnects και debugging εργαλεία [10].

Τέλος, για την υποστήριξη χαρτών και γεωχωρικών δεδομένων χρησιμοποιήθηκε η Leaflet μέσω της React-Leaflet. Η επιλογή αυτή επέτρεψε την εύκολη απόδοση διαδραστικών χαρτών και την τοποθέτηση markers ανά αισθητήρα με δυνατότητα tooltips, popups και conditional styling. Η βιβλιοθήκη είναι ελαφριά, ανοιχτού κώδικα και ευρέως διαδεδομένη, καθιστώντας την κατάλληλη για responsive εφαρμογές.

Η τελική επιλογή του τεχνολογικού stack στηρίχθηκε στην προτεραιότητα της ευελιξίας, της συντήρησης και της πλήρους λειτουργικότητας της εφαρμογής, λαμβάνοντας υπόψη τις αρχές σχεδιασμού σύγχρονων web-based IoT συστημάτων [11].

3.5 Ασφάλεια και Επεκτασιμότητα της Υλοποίησης

Η ασφάλεια και η επεκτασιμότητα αποτέλεσαν δύο από τις πιο κρίσιμες παραμέτρους κατά τον σχεδιασμό και την υλοποίηση της εφαρμογής, καθώς προορίζεται για διαχείριση ευαίσθητων δεδομένων από αισθητήρες και πρόσβαση από πολλαπλούς χρήστες σε πραγματικό χρόνο. Ιδιαίτερη μέριμνα δόθηκε τόσο στην προστασία προσωπικών δεδομένων όσο και στη σταθερή και κλιμακούμενη λειτουργία του συστήματος.

Η ασφάλεια του συστήματος εφαρμόζεται σε διάφορα επίπεδα. Κατά την αποθήκευση κωδικών χρηστών στη MySQL, χρησιμοποιείται η βιβλιοθήκη `bcrypt`, η οποία υλοποιεί αλγόριθμο hashing με salt για να αποτρέψει προσδιορισμό των πραγματικών κωδικών ακόμα και αν διαρρεύσει η βάση δεδομένων. Όλα τα αιτήματα login/register υλοποιούνται μέσω ασφαλούς POST routing, ενώ υπάρχει έλεγχος για SQL injections και validation σε backend και frontend επίπεδο. Η χρήση μεταβλητών περιβάλλοντος (`dotenv`) προστατεύει ευαίσθητες πληροφορίες όπως API keys και διαπιστευτήρια βάσης, καθώς δεν εκτίθενται ποτέ στον client [12].

Επιπλέον, εφαρμόζονται πολιτικές CORS (Cross-Origin Resource Sharing) ώστε να επιτρέπεται πρόσβαση μόνο από τα επιτρεπόμενα origins, ενώ η επικοινωνία σε παραγωγικό περιβάλλον μπορεί να υποστηρίξει HTTPS για την κρυπτογράφηση των δεδομένων. Η δομή του backend επιτρέπει την υλοποίηση middleware ελέγχου ταυτότητας σε κάθε route, αποτρέποντας μη εξουσιοδοτημένη πρόσβαση σε προστατευμένα δεδομένα ή endpoints.

Σε επίπεδο επεκτασιμότητας, η εφαρμογή έχει σχεδιαστεί modular: κάθε λειτουργία, αισθητήρας ή handler είναι απομονωμένος σε δικό του αρχείο, ακολουθώντας τη λογική separation of concerns. Αυτό σημαίνει ότι η προσθήκη ενός νέου τύπου αισθητήρα ή εξωτερικού API μπορεί να γίνει εύκολα, χωρίς να επηρεαστεί ο κεντρικός κορμός της εφαρμογής. Η χρήση Firebase ως real-time βάση δεδομένων επιτρέπει την ταυτόχρονη σύνδεση από πολλούς χρήστες χωρίς απώλεια απόδοσης, ενώ η React εξασφαλίζει ελαφρύ και αποδοτικό frontend rendering ανεξαρτήτως αριθμού ενεργών στοιχείων στο UI [13].

Το μοντέλο δεδομένων υποστηρίζει λογική ομαδοποίησης ανά χρήστη, με διαδρομές τύπου `/users/{userId}/sensors/{sensorId}`. Αυτό επιτρέπει όχι μόνο απομόνωση των δεδομένων μεταξύ χρηστών, αλλά και τη δυνατότητα μελλοντικής επέκτασης του συστήματος σε ρόλους (π.χ. admin, supervisor, operator), χωρίς αλλαγές στη δομή της βάσης. Επιπλέον, η εφαρμογή έχει σχεδιαστεί ώστε να μπορεί να εγκατασταθεί σε τοπικό server, cloud περιβάλλον (όπως Heroku, Vercel ή Firebase Hosting) ή ακόμα και σε edge συσκευές τύπου Raspberry Pi για απομακρμένη επεξεργασία.

Τέλος, η υποδομή έχει προβλέψει ενσωμάτωση τεχνολογιών τεχνητής νοημοσύνης και αυτόματης ανάλυσης μελλοντικά, είτε μέσω microservices είτε μέσω client-side libraries, προσφέροντας ένα έτοιμο πλαίσιο για εξελιγμένες λειτουργίες. Έτσι, το τελικό αποτέλεσμα είναι ένα σύστημα ταυτόχρονα ασφαλές, ευέλικτο και έτοιμο να

αναπτυχθεί περαιτέρω με βάση τις απαιτήσεις του χρήστη ή του εκάστοτε βιομηχανικού περιβάλλοντος [14].

4 Υλοποίηση Εφαρμογής

Η εφαρμογή που αναπτύχθηκε αποτελεί ένα πλήρως λειτουργικό πληροφοριακό σύστημα συλλογής, αποθήκευσης, απεικόνισης και διαχείρισης δεδομένων από βιομηχανικούς αισθητήρες, με δυνατότητες διασύνδεσης μέσω πρωτοκόλλων IoT. Το σύστημα βασίζεται σε αρχιτεκτονική client-server και είναι υλοποιημένο με σύγχρονες τεχνολογίες του οικοσυστήματος JavaScript (React στο frontend, Node.js/Express στο backend), ενώ η αποθήκευση δεδομένων γίνεται με χρήση Firebase και MySQL.

4.1 Δημιουργία και Αρχικοποίηση Έργου React

Η υλοποίηση της εφαρμογής ξεκίνησε με τη δημιουργία του frontend μέρους μέσω της πλατφόρμας React.js, η οποία επιλέχθηκε για την υψηλή απόδοση, την επεκτασιμότητα και την ευρεία υποστήριξη που προσφέρει η κοινότητά της [53]. Η αρχικοποίηση πραγματοποιήθηκε με τη χρήση του εργαλείου `npx create-react-app`, το οποίο προσφέρει μια έτοιμη δομή έργου, προρυθμισμένο περιβάλλον ανάπτυξης και αυτόματη ενσωμάτωση Webpack και Babel. Με την εκτέλεση της εντολής `npx create-react-app sensor-app`, δημιουργήθηκε ο βασικός φάκελος του έργου, μέσα στον οποίο οργανώθηκαν τα βασικά αρχεία `App.js`, `index.js`, καθώς και οι φάκελοι `public` και `src`.

Κατά τη διάρκεια της υλοποίησης, ιδιαίτερη προσοχή δόθηκε στη σαφή οργάνωση των αρχείων, ώστε ο κώδικας να είναι επαναχρησιμοποιήσιμος και ευκολότερα συντηρήσιμος. Ο φάκελος `src/components` περιλαμβάνει όλα τα ανεξάρτητα και επαναχρησιμοποιήσιμα στοιχεία διεπαφής, όπως οι φόρμες `Login`, `Register`, το `Dashboard`, οι αισθητήρες, οι πίνακες ελέγχου και τα γραφήματα. Ο φάκελος `src/pages` χρησιμοποιείται για τη διαχείριση των `views` της εφαρμογής, εξασφαλίζοντας την καθαρότητα μεταξύ των επιμέρους τμημάτων. Το αρχείο `App.js` αποτελεί τον κεντρικό δρομολογητή (`router`) και το σημείο εισόδου της λογικής πλοήγησης μέσω της βιβλιοθήκης `react-router-dom`, η οποία επιτρέπει την υλοποίηση `Single Page Application (SPA)` αρχιτεκτονικής με διακριτές διαδρομές χωρίς επαναφόρτωση σελίδας [54].

Για την απόδοση και τον επαγγελματικό σχεδιασμό του UI χρησιμοποιήθηκε η βιβλιοθήκη `@mui/material` (`Material UI`), η οποία προσφέρει ένα σύνολο από έτοιμα και `responsive UI components`. Παράλληλα, το `App.css` εμπλουτίστηκε με `custom` στυλ και `responsive` κανόνες, βασισμένους σε `CSS Grid` και `Flexbox`, ώστε η εφαρμογή να είναι πλήρως λειτουργική τόσο σε επιτραπέζιες όσο και σε κινητές συσκευές. Για καλύτερη εμπειρία χρήσης, υποστηρίζεται επίσης `dark mode`, με βάση προσαρμοσμένα θέματα και `media queries`.

Στο αρχικό στάδιο εγκαταστάθηκαν και άλλες βασικές βιβλιοθήκες που χρησιμοποιούνται ευρέως στην υλοποίηση: το axios, για την αποστολή ασφαλών HTTP αιτημάτων προς το backend [55], καθώς και το react-chartjs-2 μαζί με το chart.js και recharts, οι οποίες είναι υπεύθυνες για τη δημιουργία διαδραστικών γραφημάτων και απεικονίσεων δεδομένων αισθητήρων σε πραγματικό χρόνο [4]. Τέλος, ιδιαίτερη έμφαση δόθηκε στην ταχύτητα απόκρισης της εφαρμογής και στην modular αρχιτεκτονική, επιτρέποντας την εύκολη μελλοντική ενσωμάτωση νέων λειτουργιών ή σελίδων χωρίς να επηρεάζεται η συνολική δομή.

Η αρχικοποίηση του έργου μέσω React και η δημιουργία της βασικής αρχιτεκτονικής αποτέλεσε το θεμέλιο πάνω στο οποίο βασίστηκαν όλα τα υπόλοιπα μέρη της εφαρμογής, εξασφαλίζοντας ευελιξία, κλιμάκωση και επαγγελματική ανάπτυξη.

4.2 Ανάλυση και Χρήση Frontend Frameworks

Η κατασκευή του frontend της εφαρμογής βασίστηκε σε ένα σύνολο σύγχρονων βιβλιοθηκών και εργαλείων του οικοσυστήματος της JavaScript, με επίκεντρο τη React.js, η οποία χρησιμοποιήθηκε ως κύριο framework για τη διαχείριση της διεπαφής χρήστη. Η React αξιοποιήθηκε για την υλοποίηση επαναχρησιμοποιήσιμων components, τη διαχείριση της κατάστασης (state management) μέσω hooks, καθώς και για την αποδοτική ενημέρωση του DOM μέσω της εικονικής DOM αρχιτεκτονικής που διαθέτει [56].

Σημαντική ενίσχυση στην ανάπτυξη του UI παρείχε η χρήση της βιβλιοθήκης Material-UI (MUI), η οποία προσφέρει ένα μεγάλο εύρος από έτοιμα components, βασισμένα στη σχεδιαστική φιλοσοφία της Google. Μέσω της MUI δημιουργήθηκαν φόρμες, πίνακες, γραμμές πλοήγησης και διαδραστικά πεδία, εξασφαλίζοντας ταυτόχρονα συνέπεια και μοντέρνα αισθητική στην εφαρμογή [5]. Η αξιοποίηση του θεματικού μηχανισμού της MUI επέτρεψε την υποστήριξη τόσο φωτεινής όσο και σκοτεινής λειτουργίας (dark/light theme) με ομοιόμορφο τρόπο.

Για την πλοήγηση μεταξύ σελίδων υλοποιήθηκε το React Router DOM, το οποίο επιτρέπει την κατασκευή Single Page Applications (SPA) με διακριτά paths και δυναμική εναλλαγή views χωρίς επαναφόρτωση. Η αρχιτεκτονική SPA είναι ιδανική για εφαρμογές όπου η ταχύτητα και η εμπειρία χρήσης είναι κρίσιμοι παράγοντες, καθώς μειώνεται ο χρόνος απόκρισης και περιορίζονται τα περιττά αιτήματα προς τον διακομιστή [6].

Η διαχείριση δεδομένων από το backend και εξωτερικά APIs υλοποιήθηκε μέσω της βιβλιοθήκης Axios. Η Axios προσφέρει έναν καθαρό και συνεπή τρόπο για την εκτέλεση HTTP αιτημάτων (GET, POST, DELETE κ.λπ.) και υποστηρίζει μηχανισμούς interception, authorization headers και timeout ρυθμίσεις, καθιστώντας την απαραίτητο εργαλείο για εφαρμογές που βασίζονται σε απομακρυσμένη λήψη δεδομένων [7].

Για την απεικόνιση δεδομένων χρησιμοποιήθηκαν συνδυαστικά οι βιβλιοθήκες Chart.js και Recharts, μέσω των React wrappers react-chartjs-2 και των native components του Recharts. Η Chart.js υποστηρίζει τύπους γραφημάτων όπως γραμμικά,

πίτας και ράβδων, ενώ η Recharts προσφέρει επιπλέον δυνατότητες διαδραστικότητας και responsive σχεδιασμού. Οι δύο αυτές βιβλιοθήκες εφαρμόστηκαν σε γραφήματα αισθητήρων που συλλέγουν δεδομένα σε πραγματικό χρόνο, επιτρέποντας στους χρήστες να αντιλαμβάνονται αμέσως αποκλίσεις ή κρίσιμες τιμές [8].

Τέλος, ένα σημαντικό στοιχείο της διεπαφής είναι η υποστήριξη χαρτογραφικών δεδομένων μέσω των React Leaflet και Leaflet. Οι βιβλιοθήκες αυτές επιτρέπουν τη δυναμική προβολή των αισθητήρων πάνω σε διαδραστικούς χάρτες, χρησιμοποιώντας γεωγραφικές συντεταγμένες. Μέσα από αυτά τα εργαλεία ο χρήστης μπορεί να βλέπει με ακρίβεια τη θέση κάθε αισθητήρα στο πεδίο, κάτι ιδιαίτερα χρήσιμο σε περιβάλλοντα όπως η έξυπνη γεωργία ή η βιομηχανική παρακολούθηση μεγάλων εγκαταστάσεων [9].

Ο συνδυασμός των παραπάνω frameworks στο frontend της εφαρμογής δεν αποτέλεσε απλώς ένα σύνολο εργαλείων, αλλά ένα ενοποιημένο τεχνολογικό οικοσύστημα που υποστήριξε τη λειτουργικότητα, την εμπειρία χρήστη και τη δυνατότητα μελλοντικής επέκτασης.

4.3 Δομή και Οργάνωση Κώδικα στο Frontend

Η οργάνωση του frontend κώδικα ακολούθησε αρχές modular ανάπτυξης, με στόχο τη βελτίωση της επεκτασιμότητας, της αναγνωσιμότητας και της δυνατότητας επαναχρησιμοποίησης των στοιχείων διεπαφής. Η βασική δομή του φακέλου src περιλαμβάνει υποφακέλους για components, σελίδες (views), styles, hooks και βοηθητικές συναρτήσεις (utils), εξασφαλίζοντας ότι κάθε κατηγορία λειτουργικότητας έχει το δικό της σαφώς καθορισμένο χώρο στον κώδικα [10].

Ο φάκελος components περιλαμβάνει όλα τα βασικά επαναχρησιμοποιήσιμα στοιχεία UI, όπως φόρμες εισόδου (Login, Register), κάρτες αισθητήρων, charts, μετρητές και κουμπιά ενεργειών. Κάθε component υλοποιείται ως ανεξάρτητο JavaScript αρχείο τύπου .jsx, συνοδευόμενο από τα αντίστοιχα αρχεία στυλ, τα οποία γράφονται είτε σε CSS είτε με χρήση του @mui/system (Emotion styling API) όπου χρειάζεται. Η χρήση μικρών και ανεξάρτητων components επιτρέπει την ευέλικτη σύνθεση πιο σύνθετων views χωρίς επανάληψη κώδικα.

Τα components συνδέονται σε views μέσω του φακέλου pages, όπου ορίζονται τα βασικά containers της εφαρμογής, όπως η αρχική σελίδα, το dashboard, η σελίδα ρυθμίσεων αισθητήρων και το προφίλ χρήστη. Κάθε σελίδα εισάγει και ενορχηστρώνει τα επιμέρους components που απαιτούνται για την εμφάνισή της, ενώ περιλαμβάνει τη βασική λογική που αφορά τη ροή δεδομένων και την επικοινωνία με το backend μέσω Axios.

Η ροή δεδομένων μεταξύ των components διαχειρίζεται μέσω των React Hooks. Η χρήση των useState, useEffect, useContext και useCallback ήταν κρίσιμη για τον διαχωρισμό της κατάστασης και τη σωστή ενημέρωση του UI. Για παράδειγμα, οι μετρήσεις αισθητήρων που συλλέγονται από το backend αποθηκεύονται σε useState μεταβλητές, ενώ τα hooks useEffect χρησιμοποιούνται για την ανάκτηση δεδομένων με την είσοδο στις αντίστοιχες σελίδες ή την αλλαγή αισθητήρα. Επιπλέον, σε σημεία

όπου υπάρχει διαμοιραζόμενη κατάσταση, όπως το authentication context, χρησιμοποιήθηκε το createContext() και το useContext() ώστε η πληροφορία του ενεργού χρήστη να είναι διαθέσιμη σε ολόκληρη την εφαρμογή χωρίς prop drilling [57].

Ο φάκελος utils περιλαμβάνει χρήσιμες συναρτήσεις όπως formatters, validators και exporters. Η συνάρτηση εξαγωγής δεδομένων σε CSV, για παράδειγμα, υλοποιείται με JavaScript και καλείται από κουμπί του UI, δίνοντας τη δυνατότητα στους χρήστες να κατεβάσουν τις μετρήσεις των αισθητήρων σε αρχείο για περαιτέρω ανάλυση. Επιπλέον, μέσα από custom hooks υλοποιήθηκαν ενέργειες όπως η περιοδική ανανέωση δεδομένων αισθητήρων με χρήση του setInterval σε συνδυασμό με το useRef, ώστε να διατηρείται σωστή διαχείριση μνήμης.

Ο φάκελος styles περιλαμβάνει τα custom global styles καθώς και responsive κανόνες. Αν και η πλειοψηφία του styling βασίστηκε στην MUI, όπου κρίθηκε απαραίτητο προστέθηκαν επιπλέον CSS media queries και Flexbox/Grid ρυθμίσεις για να διασφαλιστεί η πλήρης συμβατότητα της εφαρμογής σε ποικίλες αναλύσεις και συσκευές. Όλες οι διαδρομές και η πλοήγηση διαχειρίζονται από το App.js, όπου μέσω της BrowserRouter δηλώνονται τα paths και οι αντίστοιχες σελίδες.

Η παραπάνω αρχιτεκτονική προσφέρει σαφή διαχωρισμό ανησυχιών (separation of concerns), καθιστώντας τη συντήρηση και την επέκταση της εφαρμογής πιο αποτελεσματική. Με αυτή τη λογική, κάθε προσθήκη νέου αισθητήρα, νέου view ή λειτουργίας γίνεται με ελάχιστη παρέμβαση στον υπάρχοντα κώδικα και με τρόπο συμβατό με τις αρχές της σύγχρονης ανάπτυξης frontend εφαρμογών.

4.4 Ενσωμάτωση Backend – Node.js, Express και API

Το backend της εφαρμογής σχεδιάστηκε με χρήση του Node.js σε συνδυασμό με το framework Express, με στόχο την υλοποίηση μιας ελαφριάς, ασύγχρονης και επεκτάσιμης αρχιτεκτονικής που μπορεί να ανταποκριθεί στις απαιτήσεις της συνεχούς ροής και επεξεργασίας δεδομένων από βιομηχανικούς αισθητήρες. Ο Node.js, ως runtime περιβάλλον βασισμένο στη μηχανή V8 της Google, επιτρέπει την εκτέλεση JavaScript στον διακομιστή και είναι ιδανικός για την ανάπτυξη εφαρμογών με πολλαπλές ταυτόχρονες συνδέσεις και real-time ανάγκες [58].

Η χρήση του Express ως framework επιτρέπει τον εύκολο ορισμό των endpoints μέσω της μεθόδου routing, την ενσωμάτωση middleware και την απλή διαχείριση αιτημάτων HTTP. Κατά την ανάπτυξη του backend, υλοποιήθηκαν RESTful endpoints που διαχειρίζονται τις βασικές λειτουργίες της εφαρμογής: είσοδο και εγγραφή χρηστών (/login, /register), καταχώριση νέων αισθητήρων (/register-sensor), λήψη και αποστολή δεδομένων (/get-sensor-data, /push-data) και εξαγωγή δεδομένων σε μορφή αρχείου (/export-sensor-data). Τα δεδομένα αποστέλλονται από το frontend με τη χρήση της Axios και υποβάλλονται σε έλεγχο και επεξεργασία στον server μέσω Express routes [59].

Ο κώδικας οργανώθηκε σε modules, διαχωρίζοντας τη λογική των routes από τη λογική των ελεγκτών (controllers) και των βοηθητικών συναρτήσεων. Αυτός ο διαχωρισμός επιτρέπει την καθαρή οργάνωση του backend και διευκολύνει τον έλεγχο σφαλμάτων, τις μελλοντικές επεκτάσεις και τη συγγραφή δοκιμών. Το middleware body-parser χρησιμοποιήθηκε για την ανάγνωση των αιτημάτων JSON, ενώ η βιβλιοθήκη cors χρησιμοποιήθηκε για την υποστήριξη των Cross-Origin Resource Sharing αιτημάτων που προέρχονται από τον frontend, διασφαλίζοντας παράλληλα την ασφάλεια με whitelist συγκεκριμένων origin [60].

Επιπλέον, οι ευαίσθητες μεταβλητές περιβάλλοντος, όπως κλειδιά APIs ή σύνδεση βάσεων δεδομένων, διαχειρίστηκαν μέσω του αρχείου .env, αξιοποιώντας τη βιβλιοθήκη dotenv. Η προσέγγιση αυτή επιτρέπει τη διαχείριση παραμέτρων σε διαφορετικά περιβάλλοντα (ανάπτυξης, παραγωγής) χωρίς σκληροκωδικοποίηση κρίσιμων πληροφοριών μέσα στον πηγαίο κώδικα [61].

Μία σημαντική πτυχή της backend αρχιτεκτονικής ήταν η διασύνδεση με πολλαπλές πηγές δεδομένων, τόσο από φυσικούς αισθητήρες όσο και από εξωτερικές APIs. Ο κώδικας έχει τη δυνατότητα να επεξεργάζεται real-time streams από MQTT brokers, να διαβάζει δεδομένα από σειριακές θύρες (SerialPort), να διασυνδέεται με PLCs μέσω Modbus/OPCUA, και να συνδέεται με εξωτερικές υπηρεσίες όπως το Weather API μέσω Axios. Κάθε μία από αυτές τις λειτουργίες υλοποιείται ως ξεχωριστό module, με τη δική της λογική σύνδεσης και παρακολούθησης δεδομένων.

Η ικανότητα του backend να διαχειρίζεται ταυτόχρονα πολλαπλές ροές αισθητήρων, να εκτελεί ασφαλή authentication requests και να υποστηρίζει real-time αποστολή και λήψη δεδομένων καθιστά το σύστημα κατάλληλο για βιομηχανικές εφαρμογές που απαιτούν αξιοπιστία, ταχύτητα και ελεγχόμενη πρόσβαση. Το Express παρέχει την απαραίτητη ευελιξία για τέτοιου είδους υλοποιήσεις, ενώ η χρήση εργαλείων του Node.js οικοσυστήματος ενίσχυσε την αποδοτικότητα και τη συντηρησιμότητα του backend.

4.5 Βάσεις Δεδομένων και Αποθήκευση Δεδομένων

Η αποθήκευση των δεδομένων της εφαρμογής πραγματοποιείται με συνδυασμό δύο διαφορετικών τύπων βάσεων δεδομένων, καθεμία εκ των οποίων επιλέχθηκε βάσει των ιδιαίτερων λειτουργικών απαιτήσεων. Η MySQL χρησιμοποιείται για τη διαχείριση των στοιχείων ταυτότητας των χρηστών και των ενεργειών αυθεντικοποίησης, ενώ η Firebase Realtime Database χρησιμοποιείται για την αποθήκευση και τη δυναμική διαχείριση των δεδομένων των αισθητήρων σε πραγματικό χρόνο. Ο συνδυασμός σχεσιακής και NoSQL βάσης δεδομένων επιτρέπει στη δομή της εφαρμογής να εκμεταλλεύεται τα πλεονεκτήματα και των δύο προσεγγίσεων [62].

Η MySQL, ως σχεσιακή βάση δεδομένων, είναι ιδανική για τη διαχείριση δεδομένων με αυστηρή δομή και σταθερές σχέσεις μεταξύ πινάκων. Ο πίνακας χρηστών περιλαμβάνει πεδία όπως username, email, password_hash, καθώς και timestamps για τη δημιουργία και τελευταία τροποποίηση. Οι συναλλαγές προς αυτή τη βάση

πραγματοποιούνται με χρήση συνδέσμου Node.js και της σχετικής βιβλιοθήκης mysql2, που επιτρέπει τη χρήση promises για ασύγχρονη επικοινωνία με τη βάση. Όλα τα δεδομένα ταυτοποίησης διαχειρίζονται με ασφάλεια μέσω hashing (με χρήση bcrypt), ενώ η βάση περιορίζεται αποκλειστικά στον ρόλο της ταυτοποίησης, αποφεύγοντας την αποθήκευση λειτουργικών δεδομένων όπως οι μετρήσεις αισθητήρων [63].

Για την κάλυψη των αναγκών real-time ενημέρωσης και διαχείρισης δυναμικών δεδομένων αισθητήρων, η εφαρμογή χρησιμοποιεί την πλατφόρμα Firebase Realtime Database της Google. Η Firebase αποτελεί NoSQL βάση δεδομένων, η οποία αποθηκεύει τα δεδομένα σε μορφή JSON και υποστηρίζει μηχανισμούς real-time συγχρονισμού. Η σύνδεση πραγματοποιείται μέσω του Firebase Admin SDK, και επιτρέπει την άμεση αποθήκευση, ανάκτηση και ενημέρωση δεδομένων αισθητήρων με χαμηλό latency και υψηλή αξιοπιστία [64].

Η επιλογή της Firebase κρίθηκε ιδιαίτερα κατάλληλη, καθώς προσφέρει πλεονεκτήματα όπως αυτόματη διαχείριση συνδέσεων, κλιμακούμενη αρχιτεκτονική και δυνατότητα για εξειδικευμένους κανόνες πρόσβασης (database security rules), οι οποίοι επιτρέπουν τον περιορισμό ανάγνωσης/εγγραφής σε δεδομένα μόνο από τον κάτοχό τους. Τα δεδομένα κάθε αισθητήρα αποθηκεύονται ως υποδομές εντός της διαδρομής /users/{userId}/sensors/{sensorId}, επιτρέποντας πλήρη απομόνωση και παραμετροποίηση ανά χρήστη.

Επιπλέον, σε κάθε εγγραφή περιλαμβάνονται πεδία όπως timestamp, value, status (π.χ. normal, warning, critical), καθώς και thresholds που ορίζονται από τον χρήστη. Αυτά τα δεδομένα χρησιμοποιούνται τόσο για την απεικόνιση σε γραφήματα όσο και για ειδοποιήσεις σε περίπτωση υπέρβασης ορίων. Οι εγγραφές μπορούν να ανακτηθούν κατά ημερομηνία ή με χρονικό φίλτράρισμα, διευκολύνοντας την παρουσίασή τους στο dashboard και την εξαγωγή σε CSV για offline επεξεργασία.

Η χρήση δύο διαφορετικών τύπων βάσεων, σχεσιακής και NoSQL, προσφέρει σημαντικά πλεονεκτήματα: από τη μία διασφαλίζεται η σταθερότητα και ασφάλεια της διαχείρισης λογαριασμών χρηστών, ενώ από την άλλη εξυπηρετείται η ανάγκη για ευέλικτη, ταχύτατη και real-time αποθήκευση μετρήσεων από πολλαπλές πηγές αισθητήρων. Το μοντέλο αυτό καθιστά την εφαρμογή εύκολα επεκτάσιμη, αξιόπιστη και κατάλληλη για βιομηχανικά σενάρια όπου η ασφάλεια δεδομένων και η ταχύτητα απόκρισης είναι καθοριστικής σημασίας.

4.6 Διασύνδεση με IoT Πρωτόκολλα και Sensor Handlers

Η ανάγκη για ενσωμάτωση διαφορετικών τύπων αισθητήρων και πρωτοκόλλων επικοινωνίας αποτέλεσε έναν από τους βασικούς άξονες υλοποίησης του backend συστήματος. Το περιβάλλον εφαρμογής απαιτούσε σύνδεση με πραγματικές συσκευές, πρωτόκολλα βιομηχανικής επικοινωνίας και εξωτερικά APIs, γεγονός που οδήγησε στην υλοποίηση εξειδικευμένων Sensor Handlers και στη χρήση IoT προτύπων όπως MQTT, Modbus, OPC UA, Serial και REST APIs.

Η σύνδεση με αισθητήρες τύπου Arduino πραγματοποιήθηκε μέσω του πρωτοκόλλου σειριακής επικοινωνίας USB (Serial Communication). Με χρήση της βιβλιοθήκης serialport του Node.js, αναπτύχθηκε handler που ανοίγει σύνδεση με τη θύρα του υπολογιστή ή του Raspberry Pi, διαβάζει τα δεδομένα που αποστέλλονται σε συγκεκριμένα διαστήματα από τον μικροελεγκτή και τα επεξεργάζεται σε πραγματικό χρόνο για να αποθηκευτούν σε βάση ή να προβληθούν στο UI. Το handler αυτό περιλαμβάνει λειτουργίες για error handling, buffer parsing και αποσύνδεση/επανασύνδεση σε περίπτωση αποτυχίας [65].

Για σύνδεση με συστήματα PLC που χρησιμοποιούν Modbus, χρησιμοποιήθηκε η βιβλιοθήκη modbus-serial, η οποία επιτρέπει την επικοινωνία τόσο με RTU όσο και με TCP συσκευές. Το backend υλοποιεί polling mechanism, μέσω του οποίου αποστέλλονται αιτήματα readHoldingRegisters σε τακτά χρονικά διαστήματα και καταχωρούνται οι τιμές από συγκεκριμένες διευθύνσεις μνήμης. Οι τιμές αυτές αναλύονται και μετατρέπονται σε κατανοητή μορφή για να προβληθούν μέσω του dashboard. Παράλληλα, υπάρχει δυνατότητα ορισμού warning και critical thresholds ώστε να σηματοδοτείται αμέσως η υπέρβαση αποδεκτών τιμών [66].

Για αισθητήρες που λειτουργούν με το πρωτόκολλο OPC UA, η βιβλιοθήκη node-opcua έδωσε τη δυνατότητα για σύνδεση με OPC servers μέσω ασφαλούς σύνδεσης, αναγνώριση διαθέσιμων nodes και εγγραφή σε συγκεκριμένες μεταβλητές (subscriptions). Η δομή των node trees που υποστηρίζει το OPC UA επιτρέπει την οργάνωση των δεδομένων σε λογικά ιεραρχικά σχήματα, κάτι που αποδίδεται πλήρως και στο περιβάλλον της εφαρμογής με χρήση JSON αναπαραστάσεων και δέντρων μεταβλητών στο UI [67].

Επιπλέον, ενσωματώθηκε διασύνδεση με το δίκτυο The Things Network (TTN), το οποίο βασίζεται στο MQTT πρωτόκολλο. Μέσω της βιβλιοθήκης mqtt και της σύνδεσης σε public broker, η εφαρμογή εγγράφεται σε topics συσκευών LoRaWAN και λαμβάνει δεδομένα μόλις αυτά αποσταλούν. Το χαρακτηριστικό αυτό είναι ιδιαίτερα χρήσιμο για απομακρυσμένους αισθητήρες που δεν έχουν άμεση ενσύρματη σύνδεση και επικοινωνούν μέσω χαμηλής ισχύος και μεγάλης εμβέλειας δικτύου (LPWAN). Οι εγγραφές πραγματοποιούνται με χρήση authentication keys και με ορισμό callback handler που καταχωρεί αυτόματα τα εισερχόμενα payloads στη βάση Firebase [68].

Τέλος, η εφαρμογή υποστηρίζει τη δυνατότητα λήψης δεδομένων από εξωτερικά APIs, όπως είναι το Weather API για τις καιρικές συνθήκες, το οποίο χρησιμοποιείται για λόγους επίδειξης αλλά και πρακτικής χρήσης, όταν ο καιρός επηρεάζει βιομηχανικές διεργασίες. Η ενσωμάτωση αυτών των APIs γίνεται μέσω Axios requests, με τις απαντήσεις να μετατρέπονται σε εσωτερικά formats και να οπτικοποιούνται ακριβώς όπως οι υπόλοιποι αισθητήρες.

Η ύπαρξη αυτής της πολυπρωτοκολικής υποδομής μετατρέπει την εφαρμογή σε μια ευέλικτη πλατφόρμα IoT παρακολούθησης, με δυνατότητα επέκτασης σε κάθε νέο αισθητήρα, ανεξαρτήτως τύπου σύνδεσης ή συστήματος μετάδοσης. Οι handlers είναι πλήρως παραμετροποιήσιμοι, και επιτρέπουν σε μελλοντικούς χρήστες να προσθέτουν νέες πηγές δεδομένων με απλό τρόπο και χωρίς ανακατασκευή του backend.

4.7 Χειρισμός Χαρτών και Γεωχωρικών Πληροφοριών

Η ενσωμάτωση γεωχωρικών δεδομένων στην εφαρμογή αποτέλεσε κρίσιμο στοιχείο για την εποπτεία αισθητήρων σε φυσικό ή βιομηχανικό χώρο, επιτρέποντας στο χρήστη να έχει σαφή εποπτεία της τοποθεσίας κάθε μονάδας μέτρησης. Η υλοποίηση χαρτών επιτεύχθηκε μέσω της χρήσης της βιβλιοθήκης Leaflet, σε συνδυασμό με το React wrapper React-Leaflet. Η επιλογή αυτών των εργαλείων βασίστηκε στη δυνατότητα απόδοσης ελαφριών και ταχέων διαδραστικών χαρτών, την υποστήριξη βασικών λειτουργιών όπως markers, popups, zooming, και την ευκολία ενσωμάτωσης σε React περιβάλλον [69].

Οι χάρτες εμφανίζονται μέσα στο dashboard της εφαρμογής, ως ξεχωριστό component, το οποίο φορτώνει πλακίδια χαρτών (tiles) από υπηρεσίες όπως OpenStreetMap. Κάθε αισθητήρας που διαθέτει γεωγραφικές συντεταγμένες (γεωγραφικό πλάτος και μήκος) αποδίδεται στον χάρτη με τη μορφή marker. Οι markers αυτοί συνοδεύονται από δυναμικά tooltips και popups που περιλαμβάνουν πληροφορίες όπως η ονομασία του αισθητήρα, η τιμή της τελευταίας μέτρησης, καθώς και η χρονική σήμανση λήψης των δεδομένων.

Κατά την καταχώριση ενός νέου αισθητήρα, ο χρήστης έχει τη δυνατότητα να ορίσει τη γεωγραφική του θέση είτε χειροκίνητα μέσω input πεδίων, είτε με κλικ πάνω στον χάρτη, οπότε και καταγράφονται αυτόματα οι συντεταγμένες. Οι συντεταγμένες αυτές αποθηκεύονται στο Firebase κάτω από το κάθε sensorId και ανακτώνται κατά την αρχικοποίηση του dashboard, εξασφαλίζοντας ότι η τοποθέτηση στον χάρτη είναι δυναμική και ανταποκρίνεται στο κάθε session.

Το σύστημα υποστηρίζει επιπλέον λειτουργίες όπως clustering για μαζική εμφάνιση πολλών markers σε μικρή περιοχή, καθώς και conditional styling, το οποίο αλλάζει το εικονίδιο του marker ανάλογα με την κατάσταση του αισθητήρα (π.χ. πράσινο για ομαλή λειτουργία, κόκκινο για κρίσιμη μέτρηση). Αυτό επιτυγχάνεται μέσω ελέγχου των thresholds που έχουν οριστεί ανά αισθητήρα και rendering του κατάλληλου εικονιδίου μέσα από custom Leaflet icons [70].

Ο χειρισμός της αλληλεπίδρασης με τον χάρτη, όπως pan και zoom, ελέγχεται μέσω state hooks στο React, επιτρέποντας την καταγραφή της θέσης προβολής ή την επικέντρωση σε συγκεκριμένο αισθητήρα μετά από ενέργεια του χρήστη. Η χρήση του useMap() hook του React-Leaflet προσφέρει πρόσβαση στον εσωτερικό χάρτη (Map object) και διευκολύνει την άμεση αλλαγή της θέσης ή του επιπέδου μεγέθυνσης.

Το γεωχωρικό επίπεδο προσθέτει λειτουργικότητα κρίσιμης σημασίας για εφαρμογές πεδίου και βιομηχανίας, όπου η γεωγραφική κατανομή των μονάδων επηρεάζει την απόδοση, τη συντήρηση και την απόκριση. Μέσω της ενσωμάτωσης Leaflet, η εφαρμογή μετατρέπεται από απλό εργαλείο παρακολούθησης σε πλήρως οπτικοποιημένο περιβάλλον διαχείρισης χώρου και αισθητήρων.

4.8 Οπτικοποίηση Δεδομένων – Γραφήματα και Μετρητές

Η οπτικοποίηση των δεδομένων των αισθητήρων αποτελεί έναν από τους βασικούς πυλώνες της εφαρμογής, καθώς προσφέρει άμεση, διαισθητική και δυναμική εικόνα για τη λειτουργία του συστήματος σε πραγματικό χρόνο. Η απόφαση για χρήση γραφημάτων και μετρητών στηρίχθηκε στην ανάγκη παρουσίασης ιστορικών και στιγμιαίων δεδομένων με τρόπο απλό αλλά ταυτόχρονα πλήρως κατανοητό στον τελικό χρήστη. Για την υλοποίηση αυτής της λειτουργίας χρησιμοποιήθηκαν κυρίως δύο ισχυρές βιβλιοθήκες: το Chart.js, μέσω του React wrapper react-chartjs-2, και η βιβλιοθήκη Recharts, η οποία είναι σχεδιασμένη ειδικά για React περιβάλλοντα [71].

Τα δεδομένα που απεικονίζονται αφορούν πραγματικές μετρήσεις από αισθητήρες, οι οποίες συλλέγονται είτε σε πραγματικό χρόνο είτε αποθηκεύονται ιστορικά. Κατά την είσοδο ενός χρήστη στην εφαρμογή και την επιλογή ενός συγκεκριμένου αισθητήρα, γίνεται αίτημα προς το backend για την ανάκτηση των αντίστοιχων δεδομένων, τα οποία επιστρέφονται ως JSON και αποθηκεύονται στο frontend state. Αυτά τα δεδομένα περνούν από preprocessing ώστε να διαχωριστούν χρονικά και να οργανωθούν κατά τρόπο που να είναι έτοιμα για χρήση στα charts.

Για την ιστορική απεικόνιση χρησιμοποιούνται γραφήματα γραμμής (line charts), τα οποία εμφανίζουν την εξέλιξη της τιμής ενός αισθητήρα μέσα στον χρόνο. Το Chart.js επιλέχθηκε για την υψηλή απόδοσή του σε μεγάλους όγκους δεδομένων, την υποστήριξη animation, την ευκολία διαμόρφωσης χρωμάτων, αξόνων και tooltips, καθώς και τη δυνατότητα zoom και pan. Οι τιμές ομαδοποιούνται με βάση χρονικά intervals (π.χ. ανά λεπτό ή ώρα) ώστε να διατηρείται η αναγνωσιμότητα των γραφημάτων [72].

Η βιβλιοθήκη Recharts χρησιμοποιείται σε περιπτώσεις όπου απαιτείται responsive σχεδιασμός ή η δημιουργία σύνθετων γραφημάτων, όπως area charts, stacked bars ή συνδυασμοί διαφορετικών τύπων. Η εύκολη ενσωμάτωση με styled-components και MUI προσφέρει πλήρη ευθυγράμμιση με το υπόλοιπο UI της εφαρμογής. Επιπλέον, υποστηρίζεται η δυναμική απόκριση στα thresholds του χρήστη: αν η τιμή ξεπερνά συγκεκριμένα όρια, το chart αλλάζει χρώμα ή εμφανίζει alert markers.

Εκτός από γραφήματα, η εφαρμογή διαθέτει και μετρητές τύπου gauge, οι οποίοι απεικονίζουν την τρέχουσα τιμή ενός αισθητήρα με τρόπο κυκλικό και διαδραστικό. Οι μετρητές αυτοί σχεδιάστηκαν ώστε να δίνουν άμεση εικόνα της κατάστασης, με αλλαγή χρώματος ανάλογα με το επίπεδο επικινδυνότητας. Για παράδειγμα, ένας αισθητήρας θερμοκρασίας που ξεπερνά τους 70°C εμφανίζεται με κόκκινο δείκτη, ενώ όταν είναι εντός φυσιολογικών ορίων παραμένει πράσινος. Η βιβλιοθήκη που χρησιμοποιήθηκε για τους gauge meters βασίζεται σε καμπύλες SVG και animation με χρήση React hooks [73].

Η οπτικοποίηση ενισχύεται με τη δυνατότητα export των δεδομένων, καθώς προσφέρεται κουμπί εξαγωγής σε μορφή CSV. Τα δεδομένα που απεικονίζονται συλλέγονται και μετατρέπονται σε κατάλληλη μορφή με JavaScript συνάρτηση που υλοποιείται στον frontend, επιτρέποντας την offline ανάλυση ή χρήση σε περιβάλλοντα όπως Excel ή Python.

Η ενσωμάτωση αυτών των μέσων οπτικοποίησης καθιστά την εφαρμογή όχι μόνο εργαλείο παρακολούθησης αλλά και πλατφόρμα λήψης αποφάσεων, καθώς οι χρήστες μπορούν να εντοπίσουν άμεσα δυσλειτουργίες ή αποκλίσεις. Τα charts και οι μετρητές ενισχύουν την κατανόηση πολύπλοκων δεδομένων και επιτρέπουν τον χειρισμό μεγάλου όγκου πληροφοριών με τρόπο λειτουργικό, αισθητικά ευχάριστο και τεχνικά αποδοτικό.

4.9 Διαχείριση Χρηστών και Ασφάλεια

Η διαχείριση χρηστών αποτελεί κομβικό σημείο της εφαρμογής, καθώς κάθε λειτουργία συνδέεται με εξατομικευμένα δεδομένα, δικαιώματα πρόσβασης και επίπεδα ασφάλειας. Η υλοποίηση ενός συστήματος αυθεντικοποίησης και προστασίας δεδομένων κρίθηκε απαραίτητη, ιδιαίτερα σε περιβάλλοντα που απαιτούν έλεγχο πρόσβασης, ευαισθησία δεδομένων και πολλαπλούς ρόλους χρήστη. Για το σκοπό αυτό, σχεδιάστηκε ένα πλήρως λειτουργικό σύστημα εγγραφής, εισόδου και διαχείρισης sessions, το οποίο ενσωματώνει σύγχρονες πρακτικές ασφάλειας και επικοινωνεί με το backend και τη βάση MySQL [74].

Κατά την εγγραφή χρήστη, το frontend συλλέγει δεδομένα όπως όνομα χρήστη, και κωδικό πρόσβασης. Πριν αποσταλούν στον server, εφαρμόζεται client-side validation ώστε να αποτραπούν βασικά σφάλματα εισόδου ή μορφοποίησης. Το αίτημα αποστέλλεται μέσω Axios σε endpoint του backend, όπου γίνεται έλεγχος μοναδικότητας και ασφαλής αποθήκευση των διαπιστευτηρίων. Ο κωδικός πρόσβασης δεν αποθηκεύεται ποτέ απευθείας, αλλά μετατρέπεται σε hash μέσω της βιβλιοθήκης bcrypt, με προσθήκη salt ώστε να ενισχύεται η προστασία έναντι επιθέσεων dictionary ή rainbow tables [75].

Κατά την είσοδο (login), ο server επαληθεύει τον συνδυασμό email και password, συγκρίνοντας το εισαγόμενο password με το αποθηκευμένο hash. Σε περίπτωση επιτυχούς σύνδεσης, δημιουργείται session token, το οποίο είτε αποστέλλεται στο frontend και αποθηκεύεται προσωρινά στη μνήμη (memory/sessionStorage), είτε διαχειρίζεται μέσω JWT (JSON Web Token) για ελεγχόμενη χρονική ισχύ και επικυρωμένη επικοινωνία μεταξύ frontend και backend. Αν και η παρούσα έκδοση χρησιμοποιεί session login, το σύστημα είναι έτοιμο να υποστηρίξει JWT-based authentication εφόσον απαιτηθεί σε μελλοντική έκδοση.

Κάθε προστατευόμενη λειτουργία της εφαρμογής (όπως καταγραφή αισθητήρα, εμφάνιση δεδομένων, εξαγωγή CSV) συνδέεται με το userId και απαιτεί έγκυρη σύνδεση. Στο backend, middleware λειτουργίες ελέγχουν την εγκυρότητα του session πριν την εκτέλεση κάθε ευαίσθητης ενέργειας, αποτρέποντας προσβάσεις από μη αυθεντικοποιημένους χρήστες. Ο διαχωρισμός χρηστών εξασφαλίζει ότι κάθε χρήστης έχει πρόσβαση μόνο στους δικούς του αισθητήρες, καθώς οι εγγραφές στη Firebase είναι δομημένες κατά /users/{userId}/sensors.

4.10 Τεχνητή Νοημοσύνη ως Εργαλείο Υποστήριξης στην Ανάπτυξη της Εφαρμογής

Κατά τη διάρκεια της ανάπτυξης της εφαρμογής, καθοριστική υπήρξε η συμβολή της τεχνητής νοημοσύνης όχι ως μέρος του συστήματος που υλοποιήθηκε, αλλά ως εργαλείο υποστήριξης του ίδιου του δημιουργού. Η χρήση προηγμένων γλωσσικών μοντέλων TN, και συγκεκριμένα του ChatGPT, βοήθησε σημαντικά σε όλα τα στάδια του σχεδιασμού, της διόρθωσης και της υλοποίησης της εργασίας, από την αρχική ιδέα μέχρι την ολοκλήρωση της εφαρμογής και την τεκμηρίωσή της.

Η αλληλεπίδραση με το μοντέλο ήταν συνεχής και άμεση. Πολλές φορές, σε στιγμές που παρουσιαζόταν σφάλμα στον κώδικα — είτε σύνταξης, είτε λογικής — η τεχνητή νοημοσύνη εντόπιζε γρήγορα το λάθος, εξηγούσε γιατί προέκυψε και πρότεινε τη διορθωμένη έκδοσή. Οι αλλαγές που σε διαφορετικές συνθήκες θα απαιτούσαν πολύωρη αναζήτηση ή πειραματισμό, επιλύονταν μέσα σε δευτερόλεπτα μέσω απλής διατύπωσης ερωτήσεων και διαλόγου.

Πέρα από την επίλυση σφαλμάτων, η συμβολή της TN επεκτάθηκε και σε πιο δημιουργικά ή σχεδιαστικά θέματα. Κατά την κατασκευή του frontend, έγιναν πολλές συζητήσεις με το μοντέλο σχετικά με εναλλακτικούς τρόπους υλοποίησης, επιλογές βιβλιοθηκών, βελτιώσεις στο user interface και αναδιατύπωση του κώδικα για καλύτερη απόδοση και σαφήνεια. Οι προτάσεις που δόθηκαν βασίζονταν τόσο σε βέλτιστες πρακτικές ανάπτυξης λογισμικού όσο και σε σύγχρονες τεχνολογικές τάσεις, καθιστώντας τη διαδικασία ουσιαστικά συνεργατική.

Η τεχνητή νοημοσύνη υπήρξε επίσης χρήσιμη στη συγγραφή του παρόντος εγγράφου. Πολλές φορές πρότεινε καλύτερες διατυπώσεις, βοήθησε στη μετάφραση τεχνικών εννοιών σε σαφή ακαδημαϊκή γλώσσα και υποστήριξε την οργάνωση των κεφαλαίων. Σε περιπτώσεις δυσκολίας στην περιγραφή τεχνικών υλοποιήσεων, η TN παρείχε άμεσα εναλλακτικά κείμενα, ενισχύοντας τη ροή και την πληρότητα του τεχνικού μέρους.

Επομένως, η συμβολή της TN στην εργασία αυτή δεν ήταν παθητική ούτε περιφερειακή. Αντιθέτως, αποτέλεσε ενεργό «συνομιλητή» στην πράξη ανάπτυξης, έναν προσωπικό βοηθό που δεν αντικατέστησε τη σκέψη και τη δημιουργικότητα του φοιτητή, αλλά την ενίσχυσε, την καθοδήγησε και την επιτάχυνε. Αυτή η μορφή συνεργασίας ανθρώπου και μηχανής αποδεικνύει στην πράξη τη δύναμη της τεχνολογίας, όταν χρησιμοποιείται με κριτική σκέψη και σωστή καθοδήγηση.

4.11 Συνολική Ενσωμάτωση Frameworks – Βιβλιοθήκες και Ρόλοι

Η ολοκλήρωση της εφαρμογής στηρίχθηκε στη στρατηγική επιλογή και ομαλή ενοποίηση ενός συνόλου από βιβλιοθήκες και frameworks, τα οποία συνεργάζονται αρμονικά ώστε να καλύψουν τις λειτουργικές, αισθητικές και τεχνικές απαιτήσεις του έργου. Η σωστή αξιοποίηση αυτών των εργαλείων επέτρεψε τη δημιουργία ενός

σύγχρονου, αποδοτικού και επεκτάσιμου συστήματος, το οποίο ανταποκρίνεται στις προδιαγραφές μιας βιομηχανικής IoT εφαρμογής με δυνατότητες real-time παρακολούθησης και διαχείρισης [76].

Στον πυρήνα του frontend βρίσκεται η React.js, η οποία προσφέρει component-based αρχιτεκτονική, διαχείριση κατάστασης και ταχύτητα απόκρισης, επιτρέποντας την κατασκευή μιας Single Page Application που παραμένει ευέλικτη και επεκτάσιμη. Η React συνεργάζεται άψογα με τη βιβλιοθήκη πλοήγησης React Router DOM, που επιτρέπει στον χρήστη να πλοηγείται μεταξύ των views χωρίς ανανέωση σελίδας. Για τη διαμόρφωση της διεπαφής και την ενιαία αισθητική εμπειρία, χρησιμοποιήθηκε το Material-UI (MUI), το οποίο προσφέρει εκατοντάδες έτοιμα components, θεματική παραμετροποίηση και υποστήριξη responsive σχεδίασης.

Η επικοινωνία μεταξύ frontend και backend επιτεύχθηκε μέσω της Axios, η οποία προσφέρει δομημένη διαχείριση HTTP αιτημάτων, με δυνατότητα για headers, error handling και συνδέσεις σε APIs. Η ανάλυση και απεικόνιση δεδομένων στο frontend πραγματοποιήθηκε μέσω του Chart.js και του Recharts, τα οποία ενσωματώθηκαν στο React περιβάλλον μέσω των wrappers react-chartjs-2 και native components αντίστοιχα, προσφέροντας δυναμικά γραφήματα, στατιστικές αναπαραστάσεις και responsive παρουσιάσεις μετρήσεων. Η React-Leaflet και η Leaflet αποτέλεσαν τον συνδυασμό που υποστήριξε την εμφάνιση διαδραστικών χαρτών και την τοποθέτηση αισθητήρων σε γεωγραφικό επίπεδο.

Στο backend, η κύρια πλατφόρμα υλοποίησης ήταν ο Node.js, ενισχυμένος με το framework Express.js, το οποίο επέτρεψε τη δημιουργία RESTful APIs και τον χειρισμό middleware. Η σύνδεση με τη σχεσιακή βάση MySQL εξασφάλισε την ασφαλή αποθήκευση και επαλήθευση των διαπιστευτηρίων των χρηστών, με χρήση της βιβλιοθήκης mysql2 και της bcrypt για την κρυπτογράφηση των κωδικών πρόσβασης. Ταυτόχρονα, η Firebase Realtime Database υποστήριξε την real-time αποθήκευση και παρακολούθηση δεδομένων αισθητήρων, με χρήση του Firebase Admin SDK και ασφαλή οργάνωση της πληροφορίας ανά χρήστη.

Οι αισθητήρες επικοινωνούν με την εφαρμογή μέσω σειράς πρωτοκόλλων. Η βιβλιοθήκη serialport επιτρέπει την αμφίδρομη επικοινωνία με Arduino συσκευές, η modbus-serial με PLCs μέσω Modbus RTU/TCP, ενώ η node-opcua διαχειρίζεται την επικοινωνία με βιομηχανικά συστήματα OPC UA. Για αισθητήρες τύπου LoRaWAN μέσω The Things Network, χρησιμοποιείται το πρωτόκολλο MQTT και η σχετική βιβλιοθήκη mqtt, εξασφαλίζοντας ελαφριά και ταχύτατη λήψη πακέτων δεδομένων από απομακρυσμένες συσκευές. Όλα τα παραπάνω διασυνδέονται με handlers που έχουν υλοποιηθεί modular, καθιστώντας το σύστημα έτοιμο να δεχτεί νέους αισθητήρες και πηγές δεδομένων.

Συνολικά, η εφαρμογή αξιοποίησε τις δυνατότητες κάθε εργαλείου στο μέγιστο δυνατό, αποφεύγοντας την περιττή πολυπλοκότητα αλλά διατηρώντας την απαραίτητη ευελιξία για επέκταση και αναβάθμιση. Το τεχνολογικό stack που επιλέχθηκε βασίστηκε σε πραγματικές ανάγκες, σε καλές πρακτικές λογισμικού και σε προοπτικές μελλοντικής ενσωμάτωσης υποδομών τεχνητής νοημοσύνης, ανάλυσης big data και αυτόματης λήψης αποφάσεων.

4.12 Ενσωμάτωση με Docker

Για την οργάνωση και την ευκολότερη εκτέλεση της εφαρμογής χρησιμοποιήθηκε η πλατφόρμα Docker. Η λογική ήταν να διαχωριστούν τα βασικά τμήματα της εφαρμογής (frontend, backend και βάση δεδομένων) σε ξεχωριστά containers, ώστε να είναι ανεξάρτητα μεταξύ τους αλλά και να επικοινωνούν με ασφαλή και ελεγχόμενο τρόπο.

Αρχικά δημιουργήθηκε ένα αρχείο **docker-compose.yml**, στο οποίο ορίστηκαν οι τρεις βασικές υπηρεσίες: η βάση δεδομένων MySQL, το backend σε Node.js και το frontend σε React. Κάθε υπηρεσία περιγράφεται με τις αντίστοιχες παραμέτρους εκκίνησης, τα ports που εκτίθενται και τους φακέλους που συνδέονται. Για τη βάση δεδομένων χρησιμοποιήθηκε η εικόνα mysql:8 και ορίστηκαν χρήστης, κωδικός και όνομα βάσης, καθώς και αρχικό script δημιουργίας πινάκων. Το backend υλοποιήθηκε ώστε να ξεκινά από το δικό του Dockerfile, το οποίο φροντίζει για την εγκατάσταση των απαραίτητων βιβλιοθηκών npm και την εκτέλεση του server. Αντίστοιχα, το frontend μεταγλωττίζεται σε παραγωγική μορφή με react-scripts και σε δεύτερο στάδιο αντιγράφεται σε ένα ελαφρύ container nginx για την εξυπηρέτηση του web περιβάλλοντος.

Η διαδικασία ανάπτυξης ξεκινά με την εντολή docker compose up --build από τον φάκελο του project. Με αυτόν τον τρόπο δημιουργούνται και εκκινούνται τα containers, ενώ γίνεται αυτόματα η σύνδεση μεταξύ τους μέσω ενός εσωτερικού δικτύου. Έτσι, το backend μπορεί να επικοινωνεί με το MySQL container χρησιμοποιώντας το όνομα της υπηρεσίας ως host, χωρίς να χρειάζεται να γνωρίζει τη διεύθυνση IP. Το frontend εξυπηρετείται στον browser στη θύρα 8080, το backend στη θύρα 5000, ενώ η βάση δεδομένων ακούει στη θύρα 3306.

Μέσα από αυτή την προσέγγιση επιτυγχάνεται πλήρης απομόνωση του περιβάλλοντος εκτέλεσης, ενώ διευκολύνεται και η φορητότητα της εφαρμογής, καθώς ολόκληρο το σύστημα μπορεί να αναπαραχθεί σε οποιονδήποτε υπολογιστή έχει εγκατεστημένο το Docker. Επιπλέον, παρέχεται ευελιξία στη διαχείριση, αφού μπορούν εύκολα να επανεκκινηθούν μεμονωμένα τα containers χωρίς να επηρεάζεται το σύνολο της εφαρμογής.

5 Πειραματική Δοκιμή του Συστήματος

5.1 Συμπεράσματα από τη Λειτουργία του Συστήματος

Η εφαρμογή που αναπτύχθηκε δοκιμάστηκε σε πραγματικές συνθήκες, με στόχο την αξιολόγηση της απόδοσης και της λειτουργικότητάς της. Κατά τη διάρκεια των δοκιμών, διαπιστώθηκε ότι το σύστημα ανταποκρίνεται επιτυχώς στις βασικές απαιτήσεις. Η σύνδεση με αισθητήρες τύπου Arduino μέσω σειριακής θύρας, TTN μέσω MQTT και PLC μέσω OPC-UA ήταν αξιόπιστη, ενώ η αποθήκευση δεδομένων στη Firebase Realtime Database πραγματοποιήθηκε χωρίς καθυστερήσεις ή απώλειες. Η απεικόνιση των μετρήσεων σε πραγματικό χρόνο, με χρήση της βιβλιοθήκης chart.js, αποδείχθηκε αποτελεσματική, ενώ οι ειδοποιήσεις για υπερβάσεις ορίων λειτουργήσαν όπως αναμενόταν. Επιπλέον, ο μηχανισμός εξαγωγής δεδομένων σε μορφή CSV λειτουργήσε επιτυχώς, επιτρέποντας την ανάλυση των δεδομένων με τρίτα εργαλεία, όπως το Excel.

5.2 Προτάσεις για Μελλοντική Εργασία

Παρόλο που το σύστημα είναι πλήρως λειτουργικό, υπάρχουν περιθώρια για βελτίωση και επέκταση. Μια σημαντική κατεύθυνση είναι η ενσωμάτωση τεχνητής νοημοσύνης (AI/ML) για την εκπαίδευση μοντέλων πρόβλεψης, τα οποία θα μπορούσαν να ανιχνεύουν ανωμαλίες στα δεδομένα των αισθητήρων ή να προβλέπουν ανάγκες συντήρησης (predictive maintenance). Επιπλέον, η υποστήριξη περισσότερων βιομηχανικών πρωτοκόλλων, όπως Profinet ή BACnet, θα μπορούσε να επεκτείνει τη διαλειτουργικότητα του συστήματος. Η χρήση βάσεων δεδομένων όπως InfluxDB ή TimescaleDB θα βελτίωνε τη διαχείριση χρονοσειρών δεδομένων, ενώ η καταγραφή γεωγραφικής θέσης των αισθητήρων και η απεικόνισή τους σε χάρτη θα ήταν χρήσιμη για εφαρμογές εκτός εγκαταστάσεων. Τέλος, ο επανασχεδιασμός του UI με πιο προηγμένα dashboards και η υλοποίηση role-based access για χρήστες διαφορετικών δικαιωμάτων θα βελτίωνε την εμπειρία χρήστη.

5.3 Προοπτικές Εξέλιξης της Εφαρμογής

Το έργο αυτό μπορεί να αποτελέσει τη βάση για την ανάπτυξη ενός πλήρους βιομηχανικού συστήματος παρακολούθησης, ικανού να συνδεθεί με πλήθος αισθητήρων, να πραγματοποιεί ανάλυση δεδομένων και να υποστηρίζει τη λήψη αποφάσεων. Πιθανές εφαρμογές περιλαμβάνουν την παρακολούθηση συνθηκών περιβάλλοντος σε βιομηχανικές μονάδες, την έξυπνη διαχείριση ενέργειας σε

εργοστάσια ή αποθήκες, καθώς και την υλοποίηση μιας πλατφόρμας SaaS για εταιρείες που επιθυμούν real-time παρακολούθηση της υποδομής τους.

6 Βιβλιογραφία

[1]React Official Documentation.

[2]Vue.js vs Angular: A Comparison. Smashing Magazine, 2022.

[3]Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing, 14(6), 80-83.

[4]Esposito, D. (2020). Modern Web Development with ASP.NET Core 3. Microsoft Press.

[5]Amazon Web Services (AWS) – Overview of Cloud Computing.
<https://aws.amazon.com/what-is-cloud-computing>

[6]Modbus Organization

[7]LoRa Alliance – What is LoRaWAN?

[8]Sauter, M. (2021). From GSM to LTE-Advanced Pro and 5G: An Introduction to Mobile Networks and Mobile Broadband. Wiley.

[9]Mahalik, N. P. (2013). Sensor Networks and Configuration: Fundamentals, Standards, Platforms, and Applications. Springer.

[10]The Things Network. HTTP Integration and MQTT API. Διαθέσιμο στο:
<https://www.thethingsnetwork.org/docs/applications/mqtt/>

[11]Richardson, L., Amundsen, M. & Ruby, S. (2013). RESTful Web APIs. O'Reilly Media.

[12]Subramanian, S. (2017). Mastering Modern Web APIs. Packt Publishing.

- [13]Google Firebase Realtime Database REST API
- [14]OpenWeatherMap API Documentation
- [15]Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- [16]Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson Education.
- [17]Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
- [18]Stonebraker, M. (2010). SQL databases v. NoSQL databases. Communications of the ACM, 53(4), 10–11.
- [19]Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Pearson Education.
- [20]Moroney, L. (2017). The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform. Apress.
- [21]Google Firebase. (2024). Firebase Documentation. Retrieved from
- [22]Shafranovich, Y. (2021). Mastering Firebase for Android Development. Packt Publishing.
- [23] Facebook. (2023). *React Documentation*.
- [24] Wieruch, R. (2022). *The Road to React*. Εκδόσεις Προγραμματιστή.
- [25] Banks, A., & Porcello, E. (2020). *React: Εφαρμογές Διεπαφών Χρήστη*. Εκδόσεις Τεχνολογίας.
- [26] Grider, S. (2021). *Μαθήματα Virtual DOM και Βελτιστοποίησης*. Online Course.
- [27] Redux Documentation. (2023). *State Management με Redux*.

- [28] Larsen, B. (2019). *React Ecosystem: Από Components έως Full-stack*. Εκδόσεις Κώδικα.
- [29] Freeman, A. (2020). *Σύγκριση Frontend Frameworks: React vs Angular vs Vue*. Διεθνές Συνέδριο Web Development.
- [30] Vercel. (2023). *Next.js: Server-Side Rendering με React*.
- [31] AWS. (2023). *AWS IoT Documentation*.
- [32] Mekki, K., et al. (2019). *Σύγκριση πρωτοκόλλων LPWAN: LoRaWAN vs NB-IoT*. Επιστημονικό Περιοδικό Δικτύων.
- [33] Gubbi, J., et al. (2020). *IoT για Έξυπνες Πόλεις*. Εκδόσεις Τεχνολογίας.
- [34] Shi, W., et al. (2021). *Edge Computing: Αρχές και Εφαρμογές*. Συνέδριο Cloud Computing.
- [35] Al-Fuqaha, A., et al. (2022). *Ενοποίηση AI και IoT*. Εκδόσεις Αναλυτικής Τεχνολογίας.
- [36] Roman, R., et al. (2021). *Ασφάλεια στο IoT: Απειλές και Λύσεις*. Επιστημονικό Περιοδικό Κυβερνοασφάλειας.
- [37] IEEE. (2023). *Πρότυπα Διαλειτουργικότητας IoT*.
- [38] Statista. (2023). *Παγκόσμια Στατιστικά IoT*.
- [39] Tanenbaum, A. S. (2021). *Δίκτυα Υπολογιστών: Αρχές και Πρωτόκολλα*. Εκδόσεις Πληροφορικής.
- [40] OPC Foundation. (2023). *OPC UA: Βιομηχανική Επικοινωνία*.
- [41] Modbus Organization. (2023). *Modbus TCP: Επίσημος Οδηγός*.
- [42] The Things Network. (2023). *MQTT & LoRaWAN Integration*.
- [43] Arduino. (2023). *Serial Communication με Arduino*.
- [44] Fielding, R. (2020). *REST API Design Best Practices*. Εκδόσεις Λογισμικού.
- [45] WeatherAPI. (2023). *Χρήση HTTP Proxy για Ασφάλεια*.
- [46] Boyes, H., et al. (2023). *SCADA Συστήματα: Αρχές και Εφαρμογές*. Εκδόσεις Βιομηχανικής Αυτοματοποίησης.

- [47] Grieves, M. (2022). *Ψηφιακοί Δίδυμοι στη Βιομηχανία 4.0*. Διεθνές Συνέδριο IoT.
- [48] Lee, J., et al. (2021). *Προγνωστική Συντήρηση με Machine Learning*. Επιστημονικό Περιοδικό Μηχανικής.
- [49] Wang, L., et al. (2023). *Υπολογιστική Όραση για Έλεγχο Ποιότητας*. Εκδόσεις Ρομποτικής.
- [50] Microsoft. (2023). *Azure IoT για Βιομηχανική Παρακολούθηση*.
- [51] ICS-CERT. (2023). *Απειλές Ασφάλειας σε Βιομηχανικά Συστήματα*.
- [52] MarketsandMarkets. (2023). *Παγκόσμια Αναφορά Βιομηχανικής Παρακολούθησης*.
- [53] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
- [54] React Router. (n.d.). *React Router DOM Documentation*.
- [55] Axios. (n.d.). *Axios HTTP client*.
- [56] React. (n.d.). *Main Concepts – React Official Documentation*.
- [57] Material UI. (n.d.). *Material UI Documentation*.
- [58] Spurlock, J. (2013). *Single Page Web Applications*. O'Reilly Media.
- [59] Axios GitHub Repository. (n.d.).
- [60] Chart.js. (n.d.). *Official Documentation*.
- [61] React Leaflet. (n.d.). *React components for Leaflet maps*.
- [62] Luttikhuisen, D. (2021). *Modular JavaScript*. Packt Publishing.
- [63] React Docs. (n.d.). *Hooks API Reference*.

- [64] Tilkov, S., & Vinoski, S. (2010). *Node.js: Using JavaScript to Build High-Performance Network Programs*. IEEE Internet Computing, 14(6), 80–83.
- [65] Express. (n.d.). *Express - Node.js web application framework*.
- [66] Mozilla Developer Network (MDN). (n.d.). *CORS Explained*.
- [67] Dotenv. (n.d.). *dotenv package*.
- [68] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- [69] MySQL Documentation. (n.d.).
- [70] Firebase. (n.d.). *Firebase Realtime Database*.
- [71] SerialPort. (n.d.). *Node.js serial port package*. [h](#)
- [72] Modbus-Serial. (n.d.). *Modbus for Node.js*.
- [73] OPC UA Foundation. (n.d.). *OPC Unified Architecture Specifications*.
- [74] The Things Network. (n.d.). *MQTT Data Integration*.
<https://www.thethingsnetwork.org/docs/applications/mqtt/>
- [75] Leaflet.js. (n.d.). *Leaflet Documentation*.
- [76] React Leaflet Icons Customization. (n.d.).

